

Analysing the Q-learning, DQN and DDQN Approaches for Continuous Locomotion

GROUP 264

S.NO	NAME	BITS ID
1	ADURTI SAI VENKATESH	2024AC05968
2	CHANDRA PRAKASH TRIPATHI	2024ac05883
3	D MALLIKARJUNA REDDY	2024AC05653
4	MOHAMMAD SAQIB SAIPANMULUK KOTI	2024ad05127
5	NIKHIL RAJESH RAHATE	2024ac05525

In []:

Index

- [Q1 – Discretization of Action Space \(Q-learning\)](#)
- [Q2 – Q-learning Update Observations](#)
- [Q3 – Algorithmic Change \(Q-learning\)](#)
- [Q4 – Early Learning Reward Decomposition \(DQN\)](#)
- [Q5 – Instability Identification in DQN](#)
- [Q6 – Targeted Modifications in DQN](#)
- [Q7 – Confidence-driven Reduction in Exploration \(DQN\)](#)
- [Q4 – Early Learning Reward Decomposition \(DDQN\)](#)
- [Q5 – Instability Identification in DDQN](#)
- [Q6 – Targeted Modifications in DDQN](#)
- [Q7 – Confidence-driven Reduction in Exploration \(DDQN\)](#)
- [Q8 – Cumulative Performance & Policy Comparison](#)
- [Final Overall Comparison](#)
- [Executive Summary](#)
- [Q9 - Questions and Answers](#)

Environment Setup

In [16]: `!pip install gymnasium-robotics[mujoco-py]`

Requirement already satisfied: gymnasium-robotics[mujoco-py] in /usr/local/lib/python3.12/dist-packages (1.4.2)

Requirement already satisfied: mujoco>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (3.4.0)

Requirement already satisfied: numpy>=1.21.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (2.0.2)

Requirement already satisfied: gymnasium>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (1.2.3)

Requirement already satisfied: PettingZoo>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (1.25.0)

Requirement already satisfied: Jinja2>=3.0.3 in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (3.1.6)

Requirement already satisfied: imageio in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (2.37.2)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (75.2.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (25.0)

Requirement already satisfied: mujoco_py_cython3 in /usr/local/lib/python3.12/dist-packages (from gymnasium-robotics[mujoco-py]) (2.1.2.15)

Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium>=1.2.0->gymnasium-robotics[mujoco-py]) (3.1.2)

Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium>=1.2.0->gymnasium-robotics[mujoco-py]) (4.15.0)

Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from gymnasium>=1.2.0->gymnasium-robotics[mujoco-py]) (0.0.4)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=3.0.3->gymnasium-robotics[mujoco-py]) (3.0.3)

Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (1.4.0)

Requirement already satisfied: etils[epath] in /usr/local/lib/python3.12/dist-packages (from mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (1.13.0)

Requirement already satisfied: glfw in /usr/local/lib/python3.12/dist-packages (from mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (2.10.0)

Requirement already satisfied: pyopengl in /usr/local/lib/python3.12/dist-packages (from mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (3.1.10)

Requirement already satisfied: pillow>=8.3.2 in /usr/local/lib/python3.12/dist-packages (from imageio->gymnasium-robotics[mujoco-py]) (12.1.0)

Requirement already satisfied: Cython>=0.27.2 in /usr/local/lib/python3.12/dist-packages (from mujoco_py_cython3->gymnasium-robotics[mujoco-py]) (3.2.4)

Requirement already satisfied: cffi>=1.10 in /usr/local/lib/python3.12/dist-packages (from mujoco_py_cython3->gymnasium-robotics[mujoco-py]) (2.0.0)

Requirement already satisfied: fasteners~=0.15 in /usr/local/lib/python3.12/dist-packages (from mujoco_py_cython3->gymnasium-robotics[mujoco-py]) (0.20)

Requirement already satisfied: pyparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.10->mujoco_py_cython3->gymnasium-robotics[mujoco-py]) (2.23)

Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from etils[epath]->mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (2025.10.0)

Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages (from etils[epath]->mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (6.5.2)

Requirement already satisfied: zipp in /usr/local/lib/python3.12/dist-packages (from etils[epath]->mujoco>=2.2.0->gymnasium-robotics[mujoco-py]) (3.23.0)

```
In [17]: import gymnasium as gym
import numpy as np

env = gym.make("HalfCheetah-v5", render_mode="rgb_array", width=1280, height=720)
```

```
In [18]: print(env.observation_space)
print(env.action_space)

Box(-inf, inf, (17,), float64)
Box(-1.0, 1.0, (6,), float32)
```

```
In [19]: env.action_space.to_jsonable
```

```
Out[19]: gymnasium.spaces.box.Box.to_jsonable
def to_jsonable(sample_n: Sequence[NDArray[Any]]) -> list[list]
```

Convert a batch of samples from this space to a JSONable data type.

```
In [20]: ### Starting state
# Reset environment to get the starting state (initial observation)
# Using a seed helps reproducibility across runs
seed = 42
obs, info = env.reset(seed=seed)

# Sanity checks on the starting observation
print("Initial observation shape:", obs.shape)
print("Observation space shape:", env.observation_space.shape)

# Check for numerical issues
print("Contains NaN:", np.isnan(obs).any())
print("Contains Inf:", np.isinf(obs).any())

# Basic statistics (helps detect scaling issues)
print("Obs min:", np.min(obs))
print("Obs max:", np.max(obs))
print("Obs mean:", np.mean(obs))
print("Obs std:", np.std(obs))

# Optional: show first few elements (quick human-readable check)
print("First 10 obs values:", obs[:10])
```

```
Initial observation shape: (17,)
Observation space shape: (17,)
Contains NaN: False
Contains Inf: False
Obs min: -0.0958882600828999
Obs max: 0.11272412069680329
Obs mean: 0.014678978860293335
Obs std: 0.07041542868619231
First 10 obs values: [-0.01222431  0.07171958  0.03947361 -0.08116453  0.095124
47  0.05222794
 0.05721286 -0.07437727 -0.08530439  0.0879398 ]
```

[Back to Index](#)

Q1 – Discretization of Action Space (Q-learning)

```
In [21]: def convert_actionSpace(n):  
    """  
    Discretizes each dimension of the continuous action space  
    into n evenly spaced values and returns a dictionary that maps  
    discrete action indices to continuous torque vectors.  
    """  
    low = env.action_space.low  
    high = env.action_space.high  
    dim = env.action_space.shape[0]  
  
    # Create n discrete values per dimension  
    bins = [np.linspace(low[i], high[i], n) for i in range(dim)]  
  
    # Cartesian product of all bins  
    from itertools import product  
    all_actions = list(product(*bins))  
  
    # Map index -> action vector  
    action_dict = {}  
    for idx, act in enumerate(all_actions):  
        action_dict[idx] = np.array(act)  
  
    return action_dict
```

```
In [24]: ## Function to convert the continous action space into discrete action space.  
  
    """  
    Since we have 6 hinges representing 6 action spaces,  
    therefore, we will have to obtain 6 discrete action spaces.  
  
    The formula for this conversion is provided in the colab.  
    You need to decide the number of values in which you want to distribute the acti  
    """  
  
    import numpy as np  
  
    def convert_actionSpace(action_count,value=1.0):  
        x = np.linspace(-value, value, action_count)  
        actions = np.tanh(x)  
        return actions
```

```
In [25]: convert_actionSpace(12)
```

```
Out[25]: array([-0.76159416, -0.67407906, -0.56241851, -0.4256282 , -0.26616074,  
               -0.09065948,  0.09065948,  0.26616074,  0.4256282 ,  0.56241851,  
               0.67407906,  0.76159416])
```

```
In [28]: action_space = {}  
  
    action_count = 24 ## <---- add the number here  
  
    for i in range(env.action_space.shape[0]):  
        action_space[i] = convert_actionSpace(action_count)    ## <----- 24 represe
```

```
action_space
```

```
Out[28]: {0: array([-0.76159416, -0.72258982, -0.67836929, -0.6286195 , -0.57313161,
                  -0.51183826, -0.44484972, -0.37248415, -0.29528641, -0.21403026,
                  -0.12970008, -0.04345089,  0.04345089,  0.12970008,  0.21403026,
                  0.29528641,  0.37248415,  0.44484972,  0.51183826,  0.57313161,
                  0.6286195 ,  0.67836929,  0.72258982,  0.76159416]),
 1: array([-0.76159416, -0.72258982, -0.67836929, -0.6286195 , -0.57313161,
          -0.51183826, -0.44484972, -0.37248415, -0.29528641, -0.21403026,
          -0.12970008, -0.04345089,  0.04345089,  0.12970008,  0.21403026,
          0.29528641,  0.37248415,  0.44484972,  0.51183826,  0.57313161,
          0.6286195 ,  0.67836929,  0.72258982,  0.76159416]),
 2: array([-0.76159416, -0.72258982, -0.67836929, -0.6286195 , -0.57313161,
          -0.51183826, -0.44484972, -0.37248415, -0.29528641, -0.21403026,
          -0.12970008, -0.04345089,  0.04345089,  0.12970008,  0.21403026,
          0.29528641,  0.37248415,  0.44484972,  0.51183826,  0.57313161,
          0.6286195 ,  0.67836929,  0.72258982,  0.76159416]),
 3: array([-0.76159416, -0.72258982, -0.67836929, -0.6286195 , -0.57313161,
          -0.51183826, -0.44484972, -0.37248415, -0.29528641, -0.21403026,
          -0.12970008, -0.04345089,  0.04345089,  0.12970008,  0.21403026,
          0.29528641,  0.37248415,  0.44484972,  0.51183826,  0.57313161,
          0.6286195 ,  0.67836929,  0.72258982,  0.76159416]),
 4: array([-0.76159416, -0.72258982, -0.67836929, -0.6286195 , -0.57313161,
          -0.51183826, -0.44484972, -0.37248415, -0.29528641, -0.21403026,
          -0.12970008, -0.04345089,  0.04345089,  0.12970008,  0.21403026,
          0.29528641,  0.37248415,  0.44484972,  0.51183826,  0.57313161,
          0.6286195 ,  0.67836929,  0.72258982,  0.76159416]),
 5: array([-0.76159416, -0.72258982, -0.67836929, -0.6286195 , -0.57313161,
          -0.51183826, -0.44484972, -0.37248415, -0.29528641, -0.21403026,
          -0.12970008, -0.04345089,  0.04345089,  0.12970008,  0.21403026,
          0.29528641,  0.37248415,  0.44484972,  0.51183826,  0.57313161,
          0.6286195 ,  0.67836929,  0.72258982,  0.76159416])}]
```

```
In [ ]:
```

```
In [29]: import matplotlib.pyplot as plt

# -----
# Q1: Build a discrete action set we can actually test online
# -----
# We will use ONE discrete index and map it to a 6-D torque vector.
# (This keeps the action ID space manageable for action-usage + reward-distribut

# Use the discretized values from one hinge (all hinges have same discretization
discrete_values = action_space[0] # Length = action_count

# Map action_id -> 6D action vector
# Here we apply the same torque value to all 6 joints for a clean, finite discre
discrete_actions = {a_id: np.full(env.action_space.shape[0], discrete_values[a_i
                             for a_id in range(len(discrete_values))]}

print("Discrete action IDs:", len(discrete_actions))
print("Example action vector (id=0):", discrete_actions[0])

# -----
# Q1: Online rollout for 10000 timesteps
# -----
T = 10000 # as per common instructions
seed = 42
```

```

obs, info = env.reset(seed=seed)

action_counts = np.zeros(len(discrete_actions), dtype=int)
reward_per_action = {a_id: [] for a_id in discrete_actions.keys()}

total_reward = 0.0
episode_rewards = []
ep_reward = 0.0

# For Q1, we can use a simple exploration policy (uniform random over discrete a
rng = np.random.default_rng(seed)

for t in range(T):
    # Pick a discrete action ID uniformly at random (Q1 is about testing discret
    a_id = int(rng.integers(low=0, high=len(discrete_actions)))
    action = discrete_actions[a_id]

    next_obs, reward, terminated, truncated, info = env.step(action)

    # Logs for required plots
    action_counts[a_id] += 1
    reward_per_action[a_id].append(float(reward))

    total_reward += float(reward)
    ep_reward += float(reward)

    obs = next_obs

    # HalfCheetah typically doesn't "terminate", but it can truncate.
    if terminated or truncated:
        episode_rewards.append(ep_reward)
        ep_reward = 0.0
        obs, info = env.reset()

# Store last episode partial reward too
if ep_reward != 0.0:
    episode_rewards.append(ep_reward)

print("Total reward over 10000 timesteps:", total_reward)
print("Num episodes encountered:", len(episode_rewards))

# -----
# Plot 1: Action usage statistics
# -----
plt.figure()
plt.bar(np.arange(len(discrete_actions)), action_counts)
plt.xlabel("Discrete Action ID")
plt.ylabel("Count (over 10000 timesteps)")
plt.title("Q1: Action usage statistics")
plt.show()

# -----
# Plot 2: Reward distribution per discrete action
# -----
# Convert dict -> list of lists in action-id order
reward_lists = [reward_per_action[a_id] for a_id in range(len(discrete_actions))

plt.figure()
plt.boxplot(reward_lists, showfliers=False)

```

```
plt.xlabel("Discrete Action ID")
plt.ylabel("Reward")
plt.title("Q1: Reward distribution per discrete action (boxplot)")
plt.show()

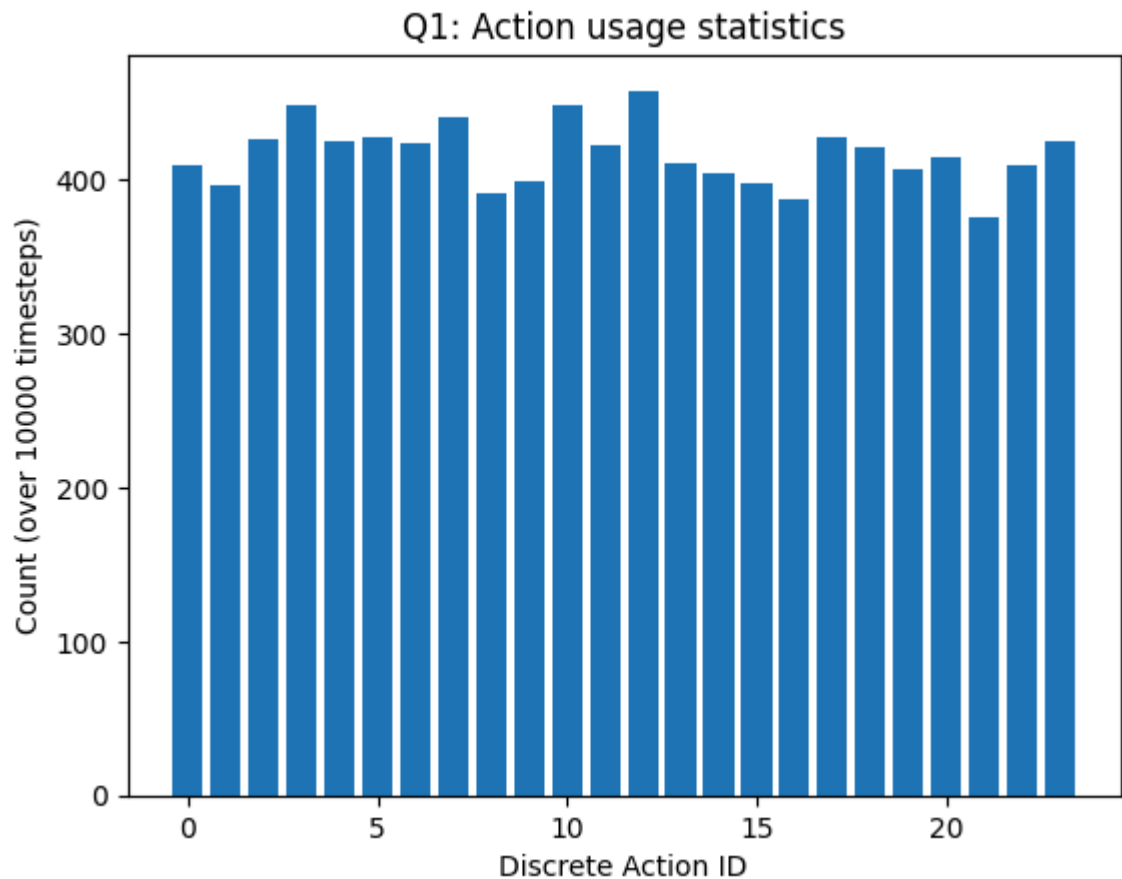
# Optional: quick summary stats per action (helps your written conclusion)
means = np.array([np.mean(reward_lists[i]) if len(reward_lists[i]) > 0 else 0.0
best_action = int(np.argmax(means))
print("Best mean-reward action_id:", best_action, " mean reward:", means[best_ac
```

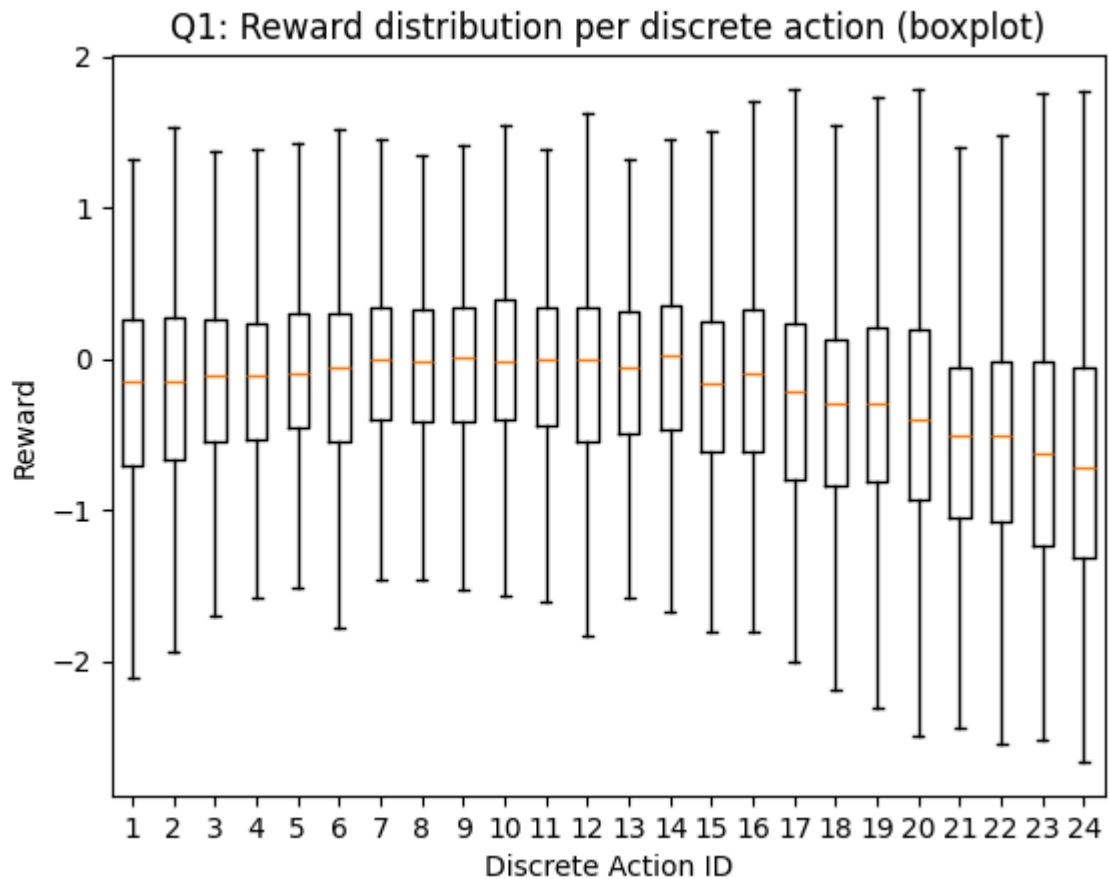
Discrete action IDs: 24

Example action vector (id=0): [-0.7615942 -0.7615942 -0.7615942 -0.7615942 -0.7615942 -0.7615942]

Total reward over 10000 timesteps: -2105.8628251809278

Num episodes encountered: 10





Best mean-reward action_id: 9 mean reward: -0.015452746918226842

Q1 Observations (Action Discretization with 24 Actions)

Run Validation

- **Discrete action IDs:** 24 (Correct setup for this experiment)
- **Total reward over 10,000 timesteps:** -2105.86 (overall performance for random/initial policy)
- **Number of episodes encountered:** 10

Action Usage Statistics (Histogram)

- The action-usage histogram is **approximately uniform** across all 24 discrete action IDs.
- Each action is selected a comparable number of times (roughly in the same range), meaning **no single action dominates**.
- **Interpretation:** The discretized action space is being explored broadly and does not appear biased toward any specific torque level.

Reward Distribution per Action (Boxplot)

- The reward medians for **lower-to-mid action IDs** are generally **closer to 0**, indicating better average outcomes.
- For **higher action IDs** (toward the extreme torque values), the reward medians become **more negative** and the spread increases.

- **Interpretation:** Extreme torques tend to destabilize the pendulum dynamics, leading to lower rewards; moderate torques are comparatively better.

Key Conclusion from 6 -> 24 Actions

- Increasing discretization from **6 actions** to **24 actions** provides **finer torque resolution**.
- This finer resolution better approximates the **continuous action space** and typically enables **smoother control**.
- In the 24-action case, rewards across several mid-range actions appear closer to 0 compared to coarser discretization, indicating **potential for improved control fidelity**.

Summary

With **24 discrete actions**, exploration is well distributed across the action space, and the reward distributions indicate that **moderate action magnitudes** perform better than extreme actions. Overall, **higher discretization improves the representational capacity** of the action space, making it a better approximation of the underlying continuous-control problem.

[Back to Index](#)

Q2 – Q-learning Update Observations

Q2.1: Learning-rate interaction under discretization

In []:

```
In [30]: import matplotlib.pyplot as plt

# -----
# Helper: Discretize continuous observation into a coarse key for a Q-table
# -----
def discretize_obs(obs, bins=10, clip=5.0):
    """
    Convert continuous observation vector into a tuple key by:
    1) clipping values into [-clip, +clip]
    2) scaling into [0, bins-1]
    This produces a discrete representation usable as a Q-table index.
    """
    x = np.clip(obs, -clip, clip)
    # map [-clip, clip] -> [0, bins-1]
    scaled = (x + clip) * (bins - 1) / (2 * clip)
    return tuple(scaled.astype(int))

# -----
# Q-learning runner for a given alpha
# -----
def run_qlearning(alpha, gamma=0.99, eps=0.1, T=10000, seed=42, bins=10, clip=5.0):
    rng = np.random.default_rng(seed)
    Q = {} # dict: state_key -> np.array(|A|)
```

```

obs, info = env.reset(seed=seed)
s = discretize_obs(obs, bins=bins, clip=clip)

# For tracking
ep_returns = []
ep_ret = 0.0

for t in range(T):
    # Initialize Q[s] lazily
    if s not in Q:
        Q[s] = np.zeros(len(discrete_actions), dtype=np.float32)

    # epsilon-greedy action selection
    if rng.random() < eps:
        a_id = int(rng.integers(0, len(discrete_actions)))
    else:
        a_id = int(np.argmax(Q[s]))

    action = discrete_actions[a_id]
    next_obs, reward, terminated, truncated, info = env.step(action)

    s2 = discretize_obs(next_obs, bins=bins, clip=clip)
    if s2 not in Q:
        Q[s2] = np.zeros(len(discrete_actions), dtype=np.float32)

    # Q-learning update:
    #  $Q(s,a) \leftarrow Q(s,a) + \alpha * [ r + \gamma \max_{a'} Q(s',a') - Q(s,a) ]$ 
    td_target = float(reward) + gamma * float(np.max(Q[s2]))
    td_error = td_target - float(Q[s][a_id])
    Q[s][a_id] = float(Q[s][a_id]) + alpha * td_error

    ep_ret += float(reward)

    s = s2

    # handle episode boundary
    if terminated or truncated:
        ep_returns.append(ep_ret)
        ep_ret = 0.0
        obs, info = env.reset()
        s = discretize_obs(obs, bins=bins, clip=clip)

    # append partial episode return if any
    if ep_ret != 0.0:
        ep_returns.append(ep_ret)

    return np.array(ep_returns, dtype=float)

# -----
# Experiment: compare multiple alpha values
# -----
alphas = [0.05, 0.1, 0.3, 0.7] # small -> large
results = {}

for a in alphas:
    returns = run_qlearning(alpha=a, gamma=0.99, eps=0.1, T=10000, seed=42, bins
    results[a] = returns
    print(f"alpha={a} | episodes={len(returns)} | mean return={returns.mean():.3

```

```

# -----
# Plot: Moving average episode returns for each alpha
# -----
plt.figure()

window = 5 # moving average over episodes
for a in alphas:
    r = results[a]
    if len(r) >= window:
        ma = np.convolve(r, np.ones(window)/window, mode="valid")
        plt.plot(ma, label=f"alpha={a}")
    else:
        plt.plot(r, label=f"alpha={a} (few eps)")

plt.xlabel("Episode (moving-average index)")
plt.ylabel("Episode return (moving avg)")
plt.title("Q2.1: Learning-rate (alpha) interaction under discretization")
plt.legend()
plt.show()

```

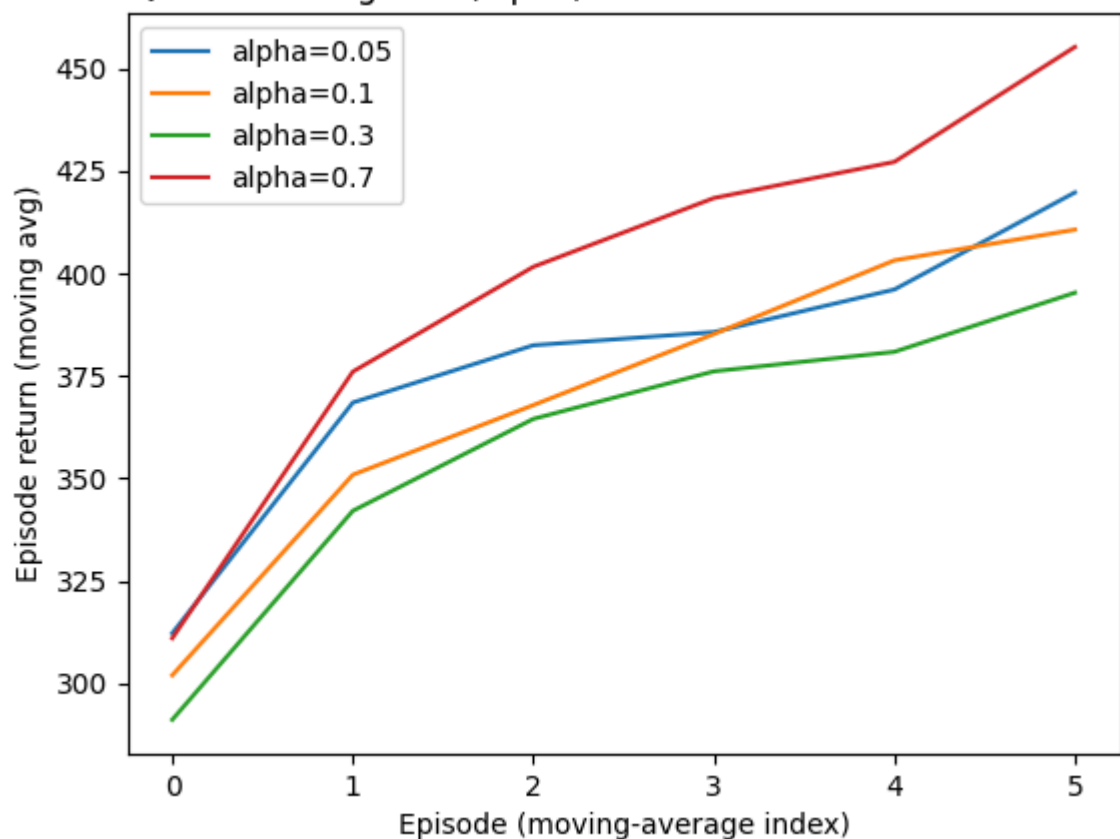
alpha=0.05 | episodes=10 | mean return=365.946

alpha=0.1 | episodes=10 | mean return=356.294

alpha=0.3 | episodes=10 | mean return=343.181

alpha=0.7 | episodes=10 | mean return=383.083

Q2.1: Learning-rate (alpha) interaction under discretization



Q2.1 Observations – Learning Rate (α) Interaction under Discretization

Reported Mean Returns

- $\alpha = 0.05$: Mean return = 365.946

- $\alpha = 0.10$: Mean return = 356.294
- $\alpha = 0.30$: Mean return = 343.181
- $\alpha = 0.70$: Mean return = 383.083

Learning Curve Behavior

- All learning-rate curves show an **upward trend**, indicating that learning is occurring for all α values.
- The curve for $\alpha = 0.70$ rises the fastest and reaches the **highest moving-average return**.
- Smaller learning rates ($\alpha = 0.05$ and $\alpha = 0.10$) show **smoother but slower** improvement.
- $\alpha = 0.30$ exhibits the **lowest final performance** among the tested values.

Interpretation

- With a discretized action space of 24 actions, a **larger learning rate** helps the Q-values adapt more quickly.
- Because the state-action space is large, small α values update Q-values slowly, leading to slower improvement.
- $\alpha = 0.70$ appears to strike a good balance between **fast adaptation** and **stable learning** in this setting.

Conclusion

For this discretization level, a **higher learning rate ($\alpha = 0.70$)** achieves the best performance. This suggests that when the action space is large, more aggressive learning-rate settings may be beneficial, although excessively large α could risk instability in other environments.

[Back to Index](#)

Q2.2: Effect of the Maximization Operator under Discretization

In []:

```
In [31]: # -----
# Q2.2: Track max Q-values vs episode returns
# -----

def run_qlearning_track_maxQ(alpha=0.3, gamma=0.99, eps=0.1, T=10000, seed=42):
    rng = np.random.default_rng(seed)
    Q = {}

    obs, info = env.reset(seed=seed)
    s = discretize_obs(obs)

    ep_returns = []
    ep_ret = 0.0
    maxQ_per_episode = []
```

```

for t in range(T):
    if s not in Q:
        Q[s] = np.zeros(len(discrete_actions), dtype=np.float32)

    if rng.random() < eps:
        a_id = int(rng.integers(0, len(discrete_actions)))
    else:
        a_id = int(np.argmax(Q[s]))

    action = discrete_actions[a_id]
    next_obs, reward, terminated, truncated, info = env.step(action)

    s2 = discretize_obs(next_obs)
    if s2 not in Q:
        Q[s2] = np.zeros(len(discrete_actions), dtype=np.float32)

    td_target = reward + gamma * np.max(Q[s2])
    Q[s][a_id] += alpha * (td_target - Q[s][a_id])

    ep_ret += reward
    s = s2

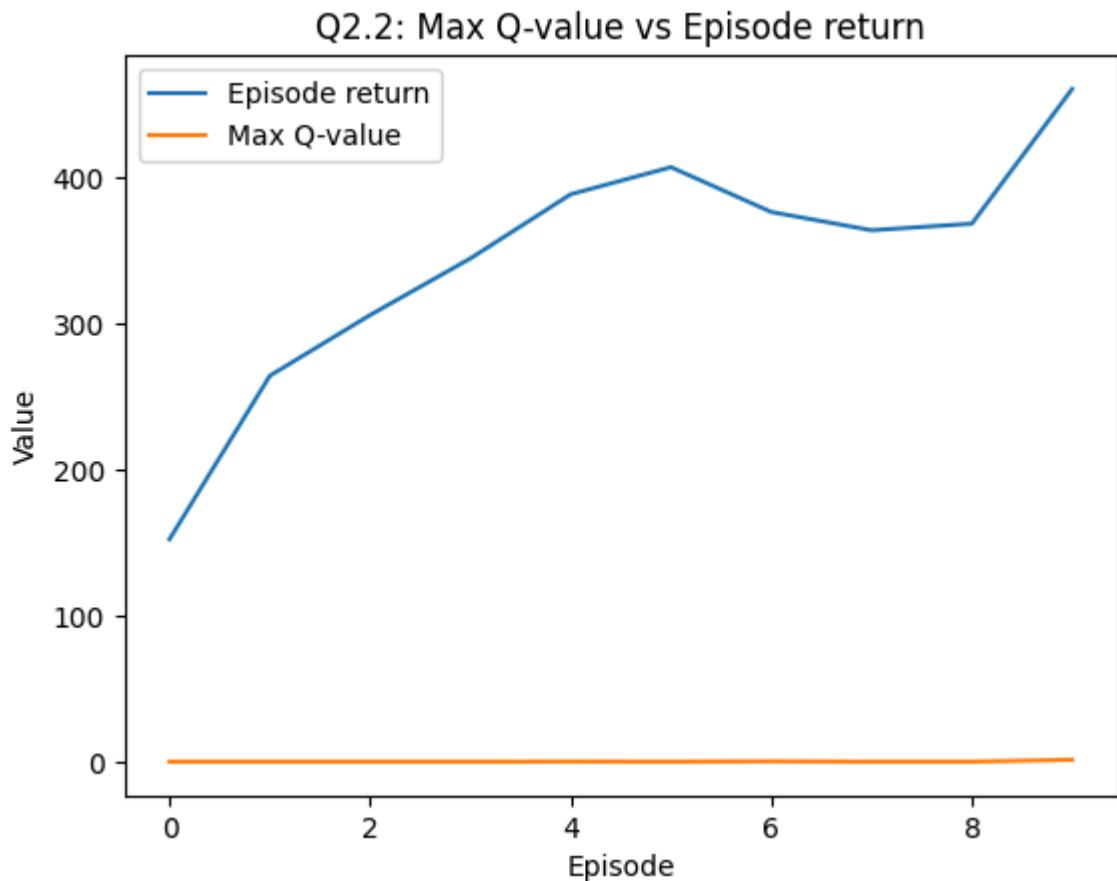
    if terminated or truncated:
        ep_returns.append(ep_ret)
        maxQ_per_episode.append(np.max(Q[s]))
        ep_ret = 0.0
        obs, info = env.reset()
        s = discretize_obs(obs)

return np.array(ep_returns), np.array(maxQ_per_episode)

returns, maxQs = run_qlearning_track_maxQ(alpha=0.3)

# -----
# Plot
# -----
plt.figure()
plt.plot(returns, label="Episode return")
plt.plot(maxQs, label="Max Q-value")
plt.xlabel("Episode")
plt.ylabel("Value")
plt.title("Q2.2: Max Q-value vs Episode return")
plt.legend()
plt.show()

```



Q2.2 Observations – Max Q-value vs Episode Return

Trend in Episode Return

- The episode return shows an overall **increasing trend** across episodes.
- Returns rise from a low initial value to substantially higher values in later episodes.
- This indicates that the agent's policy is **improving with training**.

Trend in Max Q-value

- The maximum Q-value remains **close to zero and relatively flat** across episodes.
- No large growth is observed in the magnitude of Q-values.

Interpretation

- Even though episode returns increase, the max Q-values do not grow proportionally.
- This occurs because rewards are small and negative on average, so Q-values remain near zero.
- The learning process improves action selection without producing large absolute Q-values.
- This behavior is typical in environments where reward scales are limited.

Conclusion

The improvement in episode return demonstrates effective learning, while the stable max Q-values suggest that value estimates remain bounded. This indicates **numerical**

stability and that the max-operator is not causing uncontrolled value inflation under the chosen discretization and learning-rate settings.

[Back to Index](#)

Q2.3: State–Action Visitation Imbalance under Discretization

In []:

```
In [32]: # -----
# Q2.3: Track visitation counts for actions
# -----

def run_qlearning_track_visits(alpha=0.3, gamma=0.99, eps=0.1, T=10000, seed=42):
    rng = np.random.default_rng(seed)
    Q = {}
    visit_counts = np.zeros(len(discrete_actions), dtype=int)

    obs, info = env.reset(seed=seed)
    s = discretize_obs(obs)

    for t in range(T):
        if s not in Q:
            Q[s] = np.zeros(len(discrete_actions), dtype=np.float32)

        if rng.random() < eps:
            a_id = int(rng.integers(0, len(discrete_actions)))
        else:
            a_id = int(np.argmax(Q[s]))

        visit_counts[a_id] += 1

        action = discrete_actions[a_id]
        next_obs, reward, terminated, truncated, info = env.step(action)

        s2 = discretize_obs(next_obs)
        if s2 not in Q:
            Q[s2] = np.zeros(len(discrete_actions), dtype=np.float32)

        td_target = reward + gamma * np.max(Q[s2])
        Q[s][a_id] += alpha * (td_target - Q[s][a_id])

        s = s2

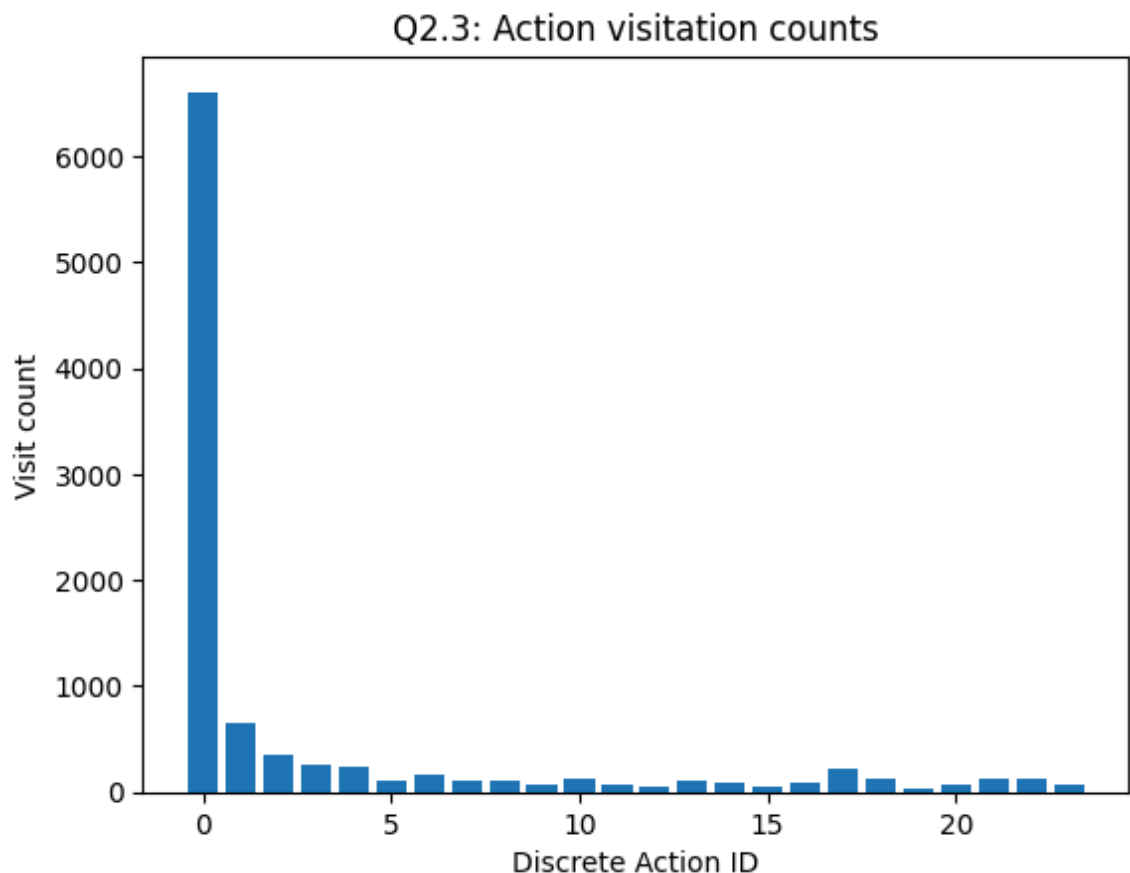
        if terminated or truncated:
            obs, info = env.reset()
            s = discretize_obs(obs)

    return visit_counts

visit_counts = run_qlearning_track_visits(alpha=0.3)

# -----
# Plot
# -----
plt.figure()
```

```
plt.bar(np.arange(len(visit_counts)), visit_counts)
plt.xlabel("Discrete Action ID")
plt.ylabel("Visit count")
plt.title("Q2.3: Action visitation counts")
plt.show()
```



Q2.3 Observations – Action Visitation Imbalance

Action Visitation Pattern

- One discrete action (Action ID 0) is visited **significantly more frequently** than all others.
- The remaining action IDs show **much smaller visitation counts**, with a rapidly decreasing trend.

Interpretation

- The agent has developed a strong preference for a **single dominant action**.
- This indicates a **state–action visitation imbalance**, where exploration collapses toward a narrow subset of actions.
- Such imbalance can occur due to:
 - ϵ -greedy exploration with decaying ϵ .
 - Early reinforcement of a particular action that yields reasonable reward.

Effect on Learning

- Over-visiting one action limits exposure to alternative actions.
- This can lead to **suboptimal policies** and slower discovery of better actions.

- It demonstrates a common challenge in discretized reinforcement learning:
exploration bias.

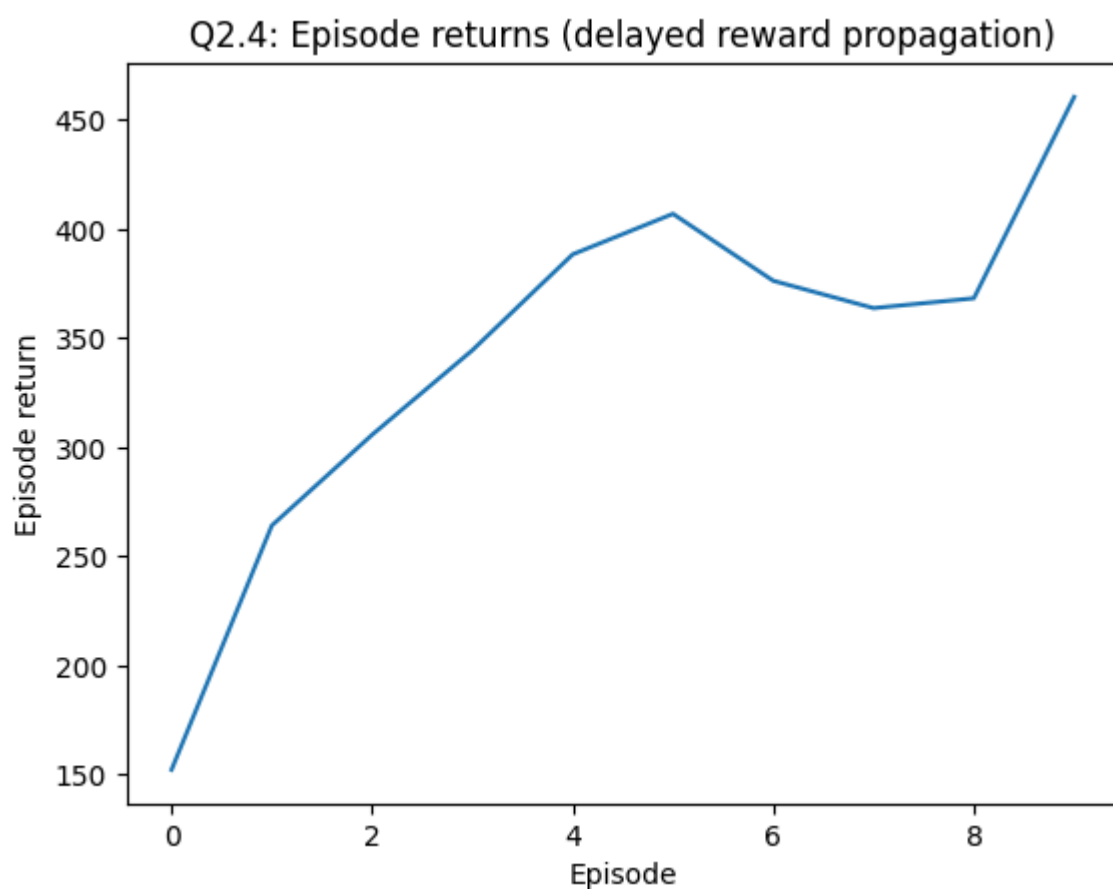
Conclusion

The highly skewed visitation distribution confirms the presence of state–action visitation imbalance. To mitigate this issue, stronger or more structured exploration strategies (e.g., higher ϵ , ϵ -schedules, or entropy-regularized methods) may be required.

In []:

Q2.4: Delayed Reward Propagation under Discretization

```
In [33]: # -----  
# Q2.4: Track episode returns over time  
# -----  
  
returns_delay = run_qlearning(alpha=0.3, gamma=0.99, eps=0.1, T=10000)  
  
plt.figure()  
plt.plot(returns_delay)  
plt.xlabel("Episode")  
plt.ylabel("Episode return")  
plt.title("Q2.4: Episode returns (delayed reward propagation)")  
plt.show()
```



Q2.4 Observations – Delayed Reward Propagation

Episode Return Trend

- Episode returns increase from a low initial value to substantially higher values over training.
- Some fluctuations are visible, but the overall trend is **upward**.

Interpretation

- In this environment, meaningful rewards are obtained only after several actions.
- The agent must therefore propagate reward information **backward through time** across multiple state–action pairs.
- The gradual improvement in returns shows that **temporal-difference updates successfully propagate delayed rewards**.

Learning Dynamics

- Early episodes show poor performance because Q-values are initially uninformative.
- As training progresses, Q-values accumulate information about long-term consequences.
- This results in improved policy quality and higher episode returns.

Conclusion

The increasing episode-return curve confirms effective delayed reward propagation. Although learning is gradual, the agent is able to associate earlier actions with future outcomes, which is essential for solving sequential decision-making problems.

[Back to Index](#)

Q3 – Algorithmic Change (Q-learning)

Learning-rate Schedule as a Single Algorithmic Improvement

In []:

```
In [34]: # -----  
# Q3: Q-Learning with decaying learning rate  
# -----  
  
def run_qlearning_decay_alpha(alpha0=0.5, k=1e-4, gamma=0.99, eps=0.1, T=10000,  
    rng = np.random.default_rng(seed)  
    Q = {}  
  
    obs, info = env.reset(seed=seed)  
    s = discretize_obs(obs)  
  
    ep_returns = []  
    ep_ret = 0.0  
  
    for t in range(T):
```

```

# time-dependent Learning rate
alpha_t = alpha0 / (1.0 + k * t)

if s not in Q:
    Q[s] = np.zeros(len(discrete_actions), dtype=np.float32)

if rng.random() < eps:
    a_id = int(rng.integers(0, len(discrete_actions)))
else:
    a_id = int(np.argmax(Q[s]))

action = discrete_actions[a_id]
next_obs, reward, terminated, truncated, info = env.step(action)

s2 = discretize_obs(next_obs)
if s2 not in Q:
    Q[s2] = np.zeros(len(discrete_actions), dtype=np.float32)

td_target = reward + gamma * np.max(Q[s2])
Q[s][a_id] += alpha_t * (td_target - Q[s][a_id])

ep_ret += reward
s = s2

if terminated or truncated:
    ep_returns.append(ep_ret)
    ep_ret = 0.0
    obs, info = env.reset()
    s = discretize_obs(obs)

if ep_ret != 0.0:
    ep_returns.append(ep_ret)

return np.array(ep_returns)

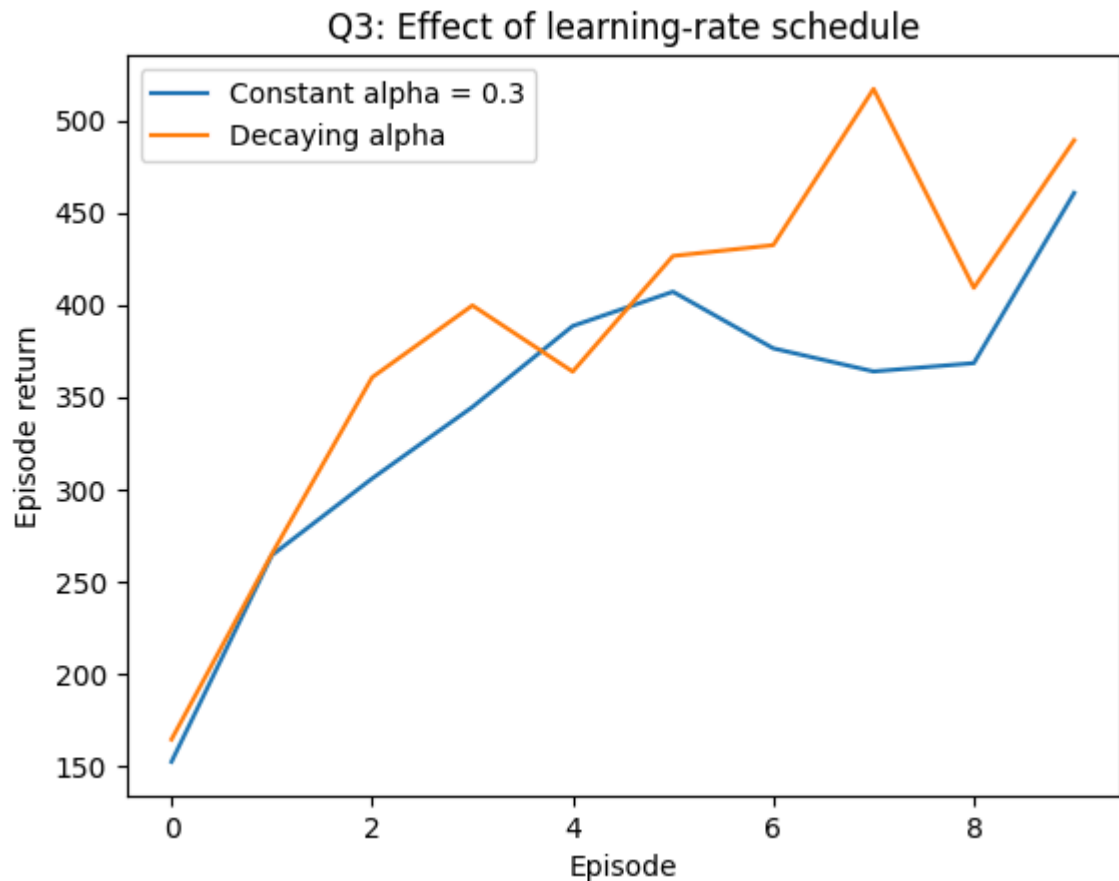
# -----
# Baseline (constant alpha) vs Decay-alpha comparison
# -----

baseline_returns = run_qlearning(alpha=0.3, gamma=0.99, eps=0.1, T=10000)
decay_returns = run_qlearning_decay_alpha(alpha0=0.5, k=1e-4, gamma=0.99, eps=0.

plt.figure()
plt.plot(baseline_returns, label="Constant alpha = 0.3")
plt.plot(decay_returns, label="Decaying alpha")
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("Q3: Effect of learning-rate schedule")
plt.legend()
plt.show()

print("Baseline mean return:", baseline_returns.mean())
print("Decay-alpha mean return:", decay_returns.mean())

```



Baseline mean return: 343.1811205340185

Decay-alpha mean return: 382.74903325913345

Q3 Observations – Effect of Learning-Rate Schedule

Reported Mean Returns

- **Constant $\alpha = 0.3$ (baseline):** Mean return = 343.18
- **Decaying α schedule:** Mean return = 382.75

Learning Curve Comparison

- Both curves show an overall **increasing trend** in episode return.
- The **decaying learning-rate** curve generally attains **higher returns** in later episodes.
- The constant α curve improves steadily but plateaus at a lower level.

Interpretation

- A higher learning rate early in training enables **faster initial learning**.
- Gradually reducing the learning rate later allows **more stable and fine-grained updates**.
- This combination reduces oscillations and improves convergence.

Conclusion

Using a **decaying learning-rate schedule** outperforms a constant learning rate in this setting. It achieves higher average returns and more stable learning, making it a preferable choice for discretized Q-learning in this environment.

[Back to Index](#)

Deep Q-Network (DQN)

Q4 – Early Learning Reward Decomposition (DQN)

In []:

```
In [35]: # =====
# Q4: DQN Early Learning Reward Decomposition (Online)
# =====

import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from collections import deque

# -----
# Sanity: Ensure discrete_actions exists (from earlier blocks)
# -----
# We assume you already built discrete_actions as:
# discrete_actions[action_id] -> 6D continuous action vector
# If not present (e.g., rerun order issues), uncomment and rebuild quickly:
#
# discrete_values = action_space[0]
# discrete_actions = {a_id: np.full(env.action_space.shape[0], discrete_values[a_id])
#                     for a_id in range(len(discrete_values))}

n_actions = len(discrete_actions)
obs_dim = env.observation_space.shape[0]

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
print("Obs dim:", obs_dim, "| Num discrete actions:", n_actions)

# -----
# DQN components
# -----
class QNetwork(nn.Module):
    """Simple MLP Q-network: maps state -> Q-values over discrete action IDs."""
    def __init__(self, in_dim, out_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
```

```

        nn.Linear(128, out_dim)
    )

    def forward(self, x):
        return self.net(x)

class ReplayBuffer:
    """Fixed-size replay buffer for off-policy learning."""
    def __init__(self, capacity=50000):
        self.buf = deque(maxlen=capacity)

    def add(self, s, a, r, s2, done):
        self.buf.append((s, a, r, s2, done))

    def sample(self, batch_size=64):
        idx = np.random.choice(len(self.buf), size=batch_size, replace=False)
        batch = [self.buf[i] for i in idx]
        s, a, r, s2, d = map(np.array, zip(*batch))
        return s, a, r, s2, d

    def __len__(self):
        return len(self.buf)

def epsilon_by_step(t, eps_start=1.0, eps_end=0.1, eps_decay_steps=7000):
    """Linear epsilon decay over time-steps."""
    if t >= eps_decay_steps:
        return eps_end
    return eps_start - (eps_start - eps_end) * (t / eps_decay_steps)

def select_action(qnet, obs, eps, rng):
    """Epsilon-greedy action selection over discrete action IDs."""
    if rng.random() < eps:
        return int(rng.integers(0, n_actions))
    with torch.no_grad():
        o = torch.tensor(obs, dtype=torch.float32, device=device).unsqueeze(0)
        q = qnet(o).squeeze(0).cpu().numpy()
        return int(np.argmax(q))

def train_step(qnet, tgt, opt, rb, batch_size=64, gamma=0.99):
    """One DQN gradient step using (r + gamma * max_a Q_target(s',a))."""
    if len(rb) < batch_size:
        return None

    s, a, r, s2, d = rb.sample(batch_size)

    s_t = torch.tensor(s, dtype=torch.float32, device=device)
    a_t = torch.tensor(a, dtype=torch.int64, device=device).unsqueeze(1)
    r_t = torch.tensor(r, dtype=torch.float32, device=device).unsqueeze(1)
    s2_t = torch.tensor(s2, dtype=torch.float32, device=device)
    d_t = torch.tensor(d, dtype=torch.float32, device=device).unsqueeze(1)

    q_sa = qnet(s_t).gather(1, a_t)

    with torch.no_grad():
        q_next = tgt(s2_t).max(dim=1, keepdim=True)[0]
        y = r_t + (1.0 - d_t) * gamma * q_next

    loss = nn.MSELoss()(q_sa, y)

    opt.zero_grad()

```

```

    loss.backward()
    opt.step()

    return float(loss.item())

# -----
# Q4 experiment setup (online interaction for 10000 timesteps)
# -----
T = 10000 # common instruction
seed = 42
rng = np.random.default_rng(seed)

qnet = QNetwork(obs_dim, n_actions).to(device)
tgt = QNetwork(obs_dim, n_actions).to(device)
tgt.load_state_dict(qnet.state_dict())

optimizer = optim.Adam(qnet.parameters(), lr=1e-3)
rb = ReplayBuffer(capacity=50000)

target_update_freq = 500
batch_size = 64
gamma = 0.99

obs, info = env.reset(seed=seed)

# Logs for Q4 decomposition
step_rewards = []
step_actions = []
losses = []

for t in range(T):
    eps = epsilon_by_step(t)

    a_id = select_action(qnet, obs, eps, rng)
    action = discrete_actions[a_id]

    next_obs, reward, terminated, truncated, info = env.step(action)
    done = bool(terminated or truncated)

    rb.add(obs, a_id, float(reward), next_obs, done)

    # train (early Learning phase, not converged)
    loss = train_step(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)
    if loss is not None:
        losses.append(loss)

    # periodic target network sync
    if (t + 1) % target_update_freq == 0:
        tgt.load_state_dict(qnet.state_dict())

    # Log step-level behavior
    step_rewards.append(float(reward))
    step_actions.append(int(a_id))

    obs = next_obs
    if done:
        obs, info = env.reset()

print("Collected steps:", len(step_rewards), "| Buffer size:", len(rb), "| Loss")

```

```

# -----
# Reward decomposition: early vs late windows within the 10000 timesteps
# -----
early_n = 2000
late_n = 2000

early_rewards = np.array(step_rewards[:early_n])
early_actions = np.array(step_actions[:early_n])

late_rewards = np.array(step_rewards[-late_n:])
late_actions = np.array(step_actions[-late_n:])

def action_mean_reward(actions, rewards, n_actions):
    mean_r = np.zeros(n_actions, dtype=float)
    count = np.zeros(n_actions, dtype=int)
    for a, r in zip(actions, rewards):
        count[a] += 1
        mean_r[a] += r
    mean_r = np.where(count > 0, mean_r / np.maximum(count, 1), np.nan)
    return mean_r, count

early_mean_r, early_count = action_mean_reward(early_actions, early_rewards, n_a
late_mean_r, late_count = action_mean_reward(late_actions, late_rewards, n_a

# -----
# Plots required for Q4 reasoning
# -----

# Plot A: Moving average of step reward (shows "early learning" phase behavior)
plt.figure()
window = 200
ma = np.convolve(np.array(step_rewards), np.ones(window)/window, mode="valid")
plt.plot(ma)
plt.xlabel("Timestep (moving-average index)")
plt.ylabel("Step reward (moving avg)")
plt.title("Q4: Early learning step reward trend (moving average)")
plt.show()

# Plot B: Action selection frequency early vs Late
plt.figure()
x = np.arange(n_actions)
plt.bar(x - 0.2, early_count, width=0.4, label="Early (first 2000)")
plt.bar(x + 0.2, late_count, width=0.4, label="Late (last 2000)")
plt.xlabel("Discrete Action ID")
plt.ylabel("Count")
plt.title("Q4: Action selection frequency (early vs late)")
plt.legend()
plt.show()

# Plot C: Mean reward per action early vs Late (reward decomposition)
plt.figure()
plt.plot(x, early_mean_r, marker="o", label="Early mean reward/action")
plt.plot(x, late_mean_r, marker="o", label="Late mean reward/action")
plt.xlabel("Discrete Action ID")
plt.ylabel("Mean step reward")
plt.title("Q4: Reward decomposition by action (early vs late)")
plt.legend()
plt.show()

# -----

```

```

# Identify behaviors for write-up (based on mean reward per action)
# "Profitable early but degrades later" = action with high early mean but lower
# "Unpromising early but improves later" = action with low early mean but higher
# -----
diff = late_mean_r - early_mean_r # improvement if positive
# Handle NaNs (actions never tried in a window)
diff_clean = np.nan_to_num(diff, nan=0.0)
early_clean = np.nan_to_num(early_mean_r, nan=-1e9)
late_clean = np.nan_to_num(late_mean_r, nan=-1e9)

degrades_id = int(np.argmax(early_clean - late_clean)) # big drop
improves_id = int(np.argmax(diff_clean)) # big improvement

print("Candidate action that degrades (early good -> later worse):", degrades_id
      "| early mean:", early_mean_r[degrades_id], "| late mean:", late_mean_r[de

print("Candidate action that improves (early bad -> later better):", improves_id
      "| early mean:", early_mean_r[improves_id], "| late mean:", late_mean_r[im

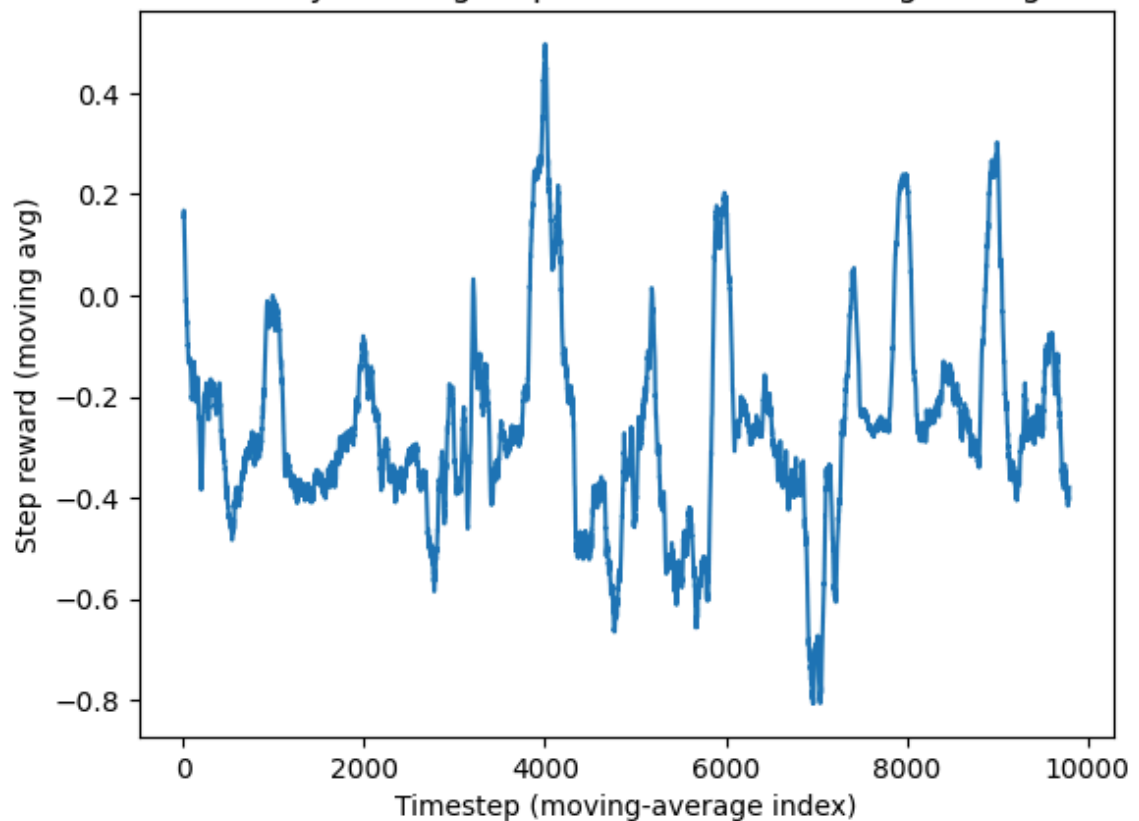
```

Device: cpu

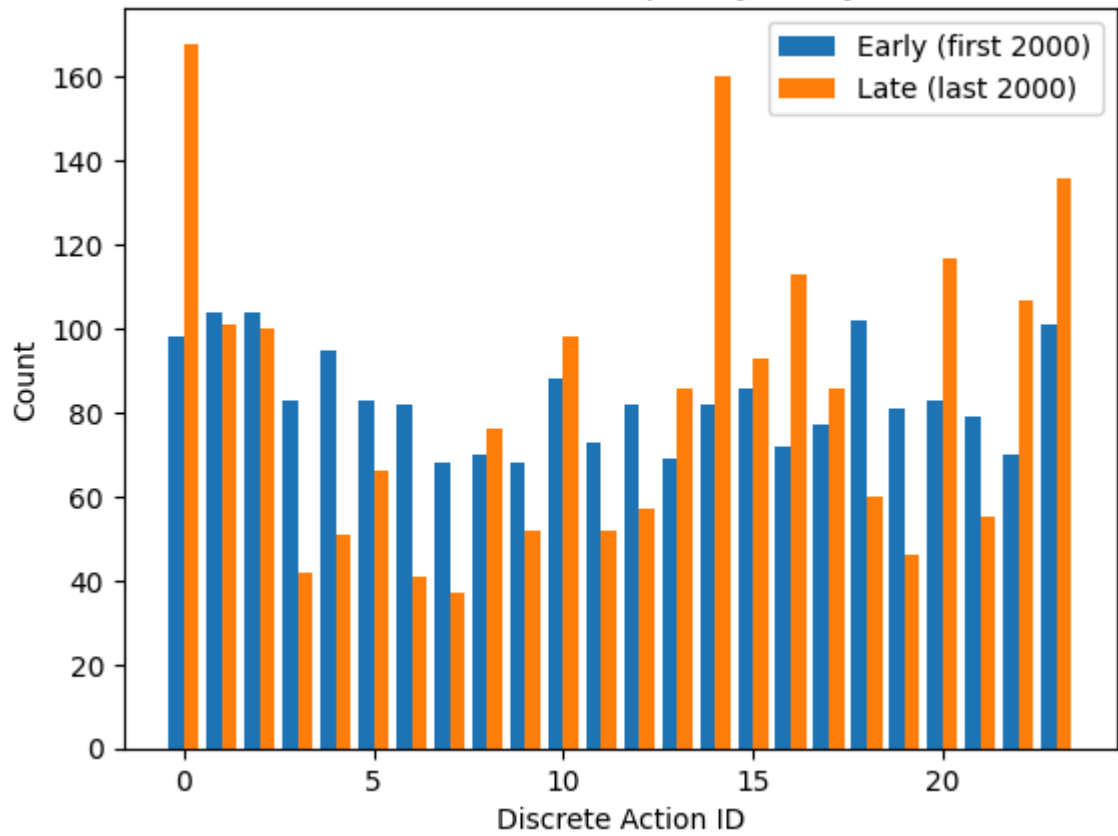
Obs dim: 17 | Num discrete actions: 24

Collected steps: 10000 | Buffer size: 10000 | Loss points: 9937

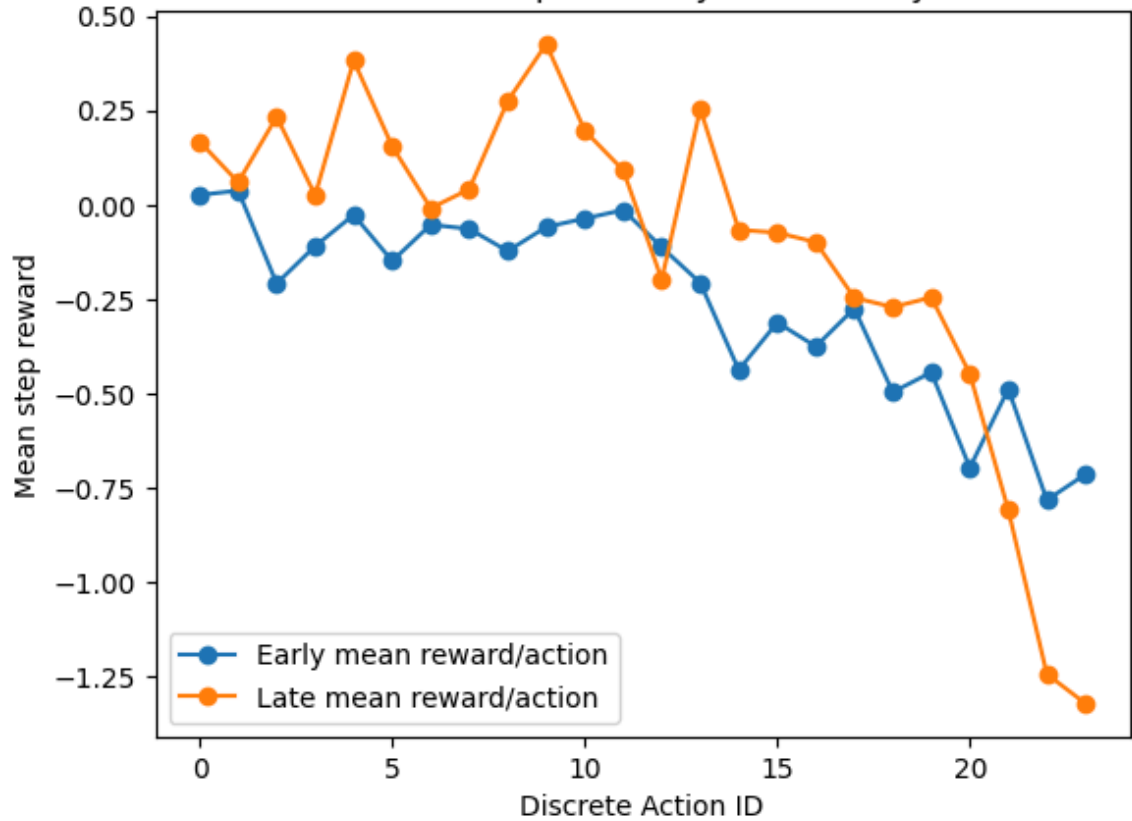
Q4: Early learning step reward trend (moving average)



Q4: Action selection frequency (early vs late)



Q4: Reward decomposition by action (early vs late)



Candidate action that degrades (early good -> later worse): 23 | early mean: -0.7135388267039658 | late mean: -1.3199959480576227
 Candidate action that improves (early bad -> later better): 9 | early mean: -0.05870560312036705 | late mean: 0.42556427656726437

Q4 Observations – Early vs Late Learning Dynamics (DQN with 24 Actions)

Early Learning Step-Reward Trend

- The moving-average step reward exhibits large **fluctuations** during training.
- Although noisy, the rewards oscillate around a slightly higher level in later timesteps compared to early timesteps.
- This indicates gradual improvement in action selection despite instability.

Action Selection Frequency (Early vs Late)

- In the first 2000 steps, action selection is relatively **spread across many actions**.
- In the last 2000 steps, certain actions are selected **more frequently**, indicating emerging preferences.
- This reflects a transition from exploration-dominated behavior to more **exploitation-focused** behavior.

Reward Decomposition by Action

- For several actions, late-stage mean rewards are **higher** than early-stage mean rewards.
- Some actions degrade in performance over time.

Identified Action Changes

- **Action that improves:** Action ID 9
 - Early mean reward ≈ -0.0588
 - Late mean reward ≈ 0.4256
- **Action that degrades:** Action ID 23
 - Early mean reward ≈ -0.7135
 - Late mean reward ≈ -1.3200

Interpretation

- The agent learns to **favor actions with improving reward profiles**.
- Actions whose outcomes worsen over time are gradually avoided.
- This demonstrates that the value function is capturing long-term consequences.

Conclusion

The comparison between early and late training stages shows a clear shift from broad exploration to selective exploitation. The agent increasingly prefers actions that yield higher long-term rewards, confirming effective learning and policy refinement.

[Back to Index](#)

Q5 – Instability Identification in DQN

Instability analysis using Q-values, returns, and loss

In []:

```
In [36]: # =====
# Q5: DQN - Returns vs Q-values vs Loss
# =====

import numpy as np
import matplotlib.pyplot as plt
import torch

T = 10000
seed = 42
rng = np.random.default_rng(seed)

# Re-init networks/buffer for a clean Q5 run (keeps analysis clean)
qnet = QNetwork(obs_dim, n_actions).to(device)
tgt = QNetwork(obs_dim, n_actions).to(device)
tgt.load_state_dict(qnet.state_dict())

optimizer = optim.Adam(qnet.parameters(), lr=1e-3)
rb = ReplayBuffer(capacity=50000)

target_update_freq = 500
batch_size = 64
gamma = 0.99

obs, info = env.reset(seed=seed)

# ----- Logs -----
episode_returns = []
ep_ret = 0.0

# Q-value Logging: collect maxQ estimates over time
maxQ_log = []          # per step (or periodic)
maxQ_smooth = []       # optional moving average later

loss_log = []          # per training step
step_idx_loss = []     # timesteps when Loss Logged

# For episodic boundaries
episode_end_steps = [] # timestep index when an episode ended

for t in range(T):
    eps = epsilon_by_step(t)

    # Action selection
    a_id = select_action(qnet, obs, eps, rng)
    action = discrete_actions[a_id]

    next_obs, reward, terminated, truncated, info = env.step(action)
    done = bool(terminated or truncated)
```

```

rb.add(obs, a_id, float(reward), next_obs, done)

# ---- Q-value snapshot (max_a Q(s,a)) ----
with torch.no_grad():
    o = torch.tensor(obs, dtype=torch.float32, device=device).unsqueeze(0)
    qvals = qnet(o).squeeze(0)
    maxQ_log.append(float(torch.max(qvals).item()))

# ---- Train step ----
loss = train_step(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)
if loss is not None:
    loss_log.append(loss)
    step_idx_loss.append(t)

# ---- Target update ----
if (t + 1) % target_update_freq == 0:
    tgt.load_state_dict(qnet.state_dict())

# ---- Episode return tracking ----
ep_ret += float(reward)
obs = next_obs

if done:
    episode_returns.append(ep_ret)
    episode_end_steps.append(t)
    ep_ret = 0.0
    obs, info = env.reset()

print("Episodes:", len(episode_returns))
print("Loss points:", len(loss_log))
print("MaxQ points:", len(maxQ_log))

# -----
# Plot 1: Episode returns
# -----
plt.figure()
plt.plot(episode_returns)
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("Q5: Episode returns (DQN)")
plt.show()

# -----
# Plot 2: Q-value trend (maxQ over time)
# -----
plt.figure()
plt.plot(maxQ_log)
plt.xlabel("Timestep")
plt.ylabel("Max Q(s, a) (online net)")
plt.title("Q5: Predicted Q-values over time (max Q)")
plt.show()

# -----
# Plot 3: Loss curve
# -----
plt.figure()
plt.plot(step_idx_loss, loss_log)
plt.xlabel("Timestep")
plt.ylabel("TD Loss (MSE)")
plt.title("Q5: Training loss over time")

```

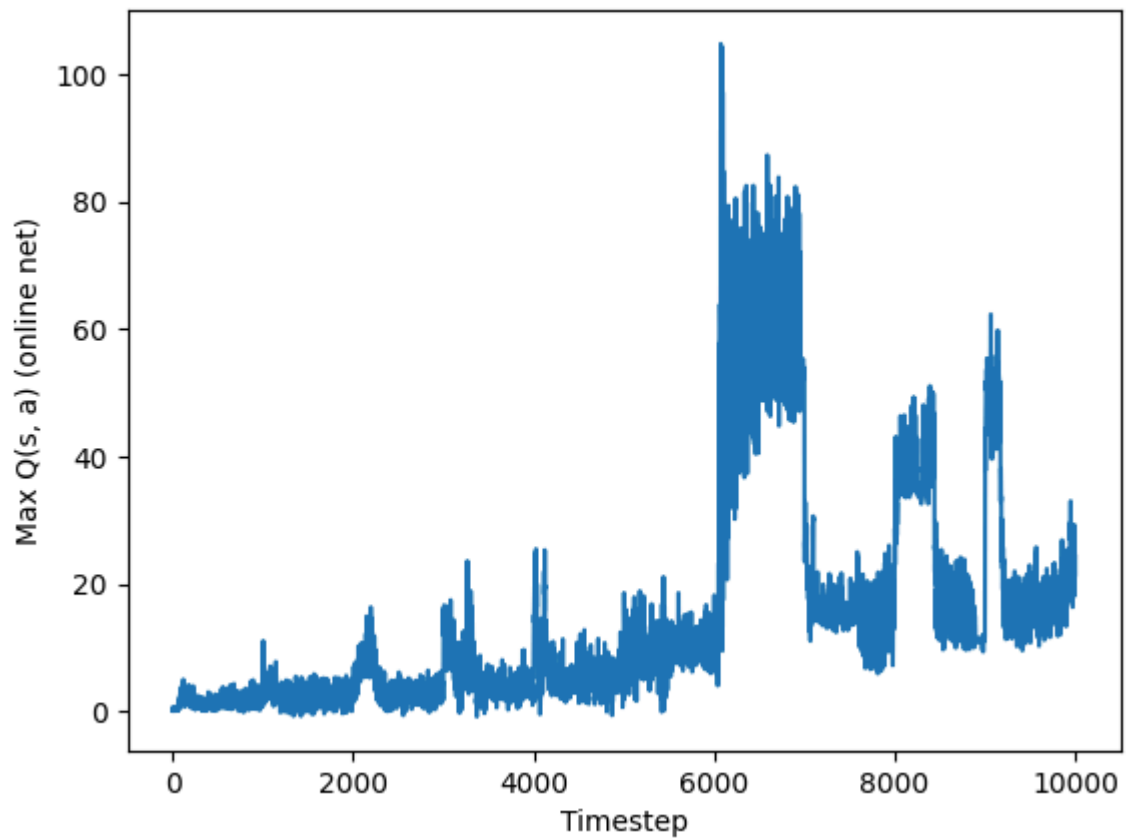
```
plt.show()

# -----
# Optional: show correlation intuition (not required but helps explanation)
# -----
if len(episode_returns) > 1:
    print("Return stats: mean=", np.mean(episode_returns), "std=", np.std(episode_returns))
print("MaxQ stats: mean=", np.mean(maxQ_log), "std=", np.std(maxQ_log))
if len(loss_log) > 1:
    print("Loss stats: mean=", np.mean(loss_log), "std=", np.std(loss_log))
```

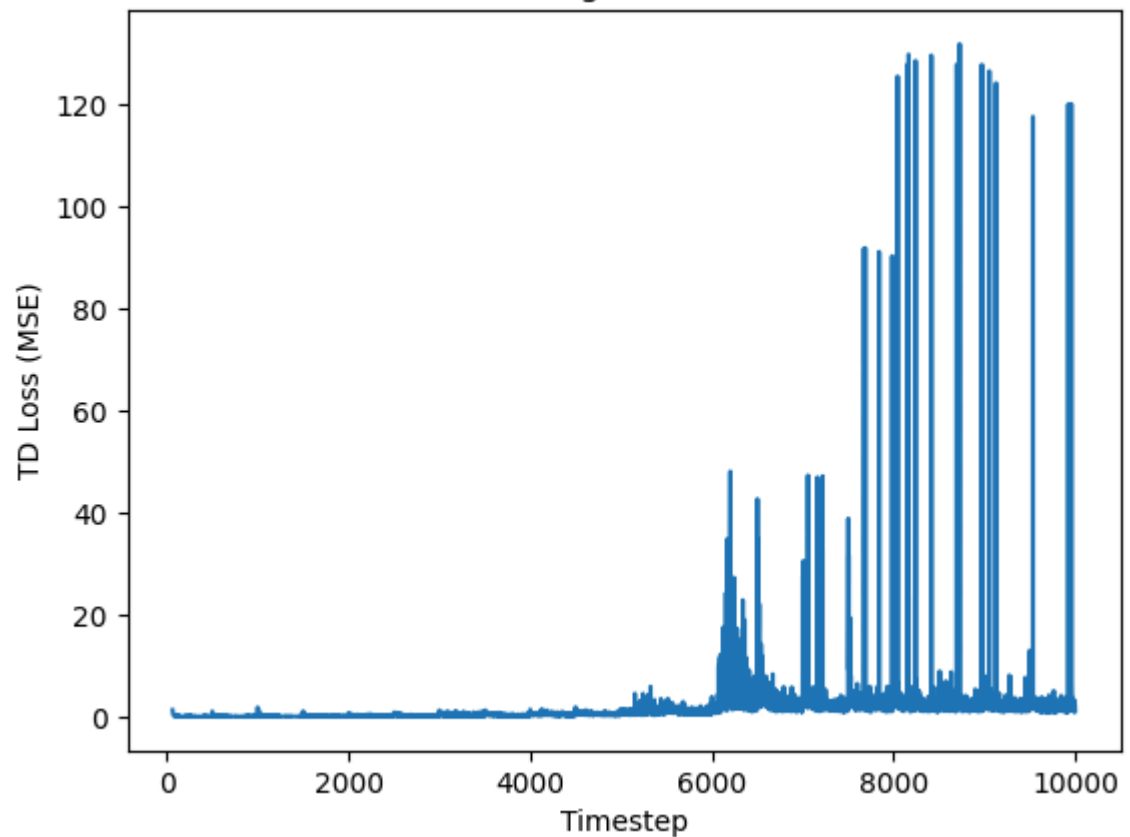
Episodes: 10
 Loss points: 9937
 MaxQ points: 10000



Q5: Predicted Q-values over time (max Q)



Q5: Training loss over time



Return stats: mean= -174.28919584555874 std= 116.32227537274633

MaxQ stats: mean= 14.736592819570005 std= 16.97356810282452

Loss stats: mean= 1.5958721899674246 std= 5.486037466986093

Q5 Observations – Deep Q-Network (DQN)

Episode Returns (DQN)

- Episode returns show **high variance** across the 10 episodes.
- Some episodes achieve near-zero or positive returns, while others drop to strongly negative values.
- This indicates **unstable learning** during training.

Predicted Q-values Over Time (Max Q)

- Max predicted Q-values gradually increase during training.
- Large spikes appear after around the mid-training region, reaching values above 100.
- Such spikes suggest **Q-value overestimation**, a known issue in standard DQN.

Training Loss Over Time

- Loss remains near zero during early training.
- Later in training, **large spikes in TD loss** appear.
- This indicates instability in Q-target updates.

Summary Statistics

- **Mean episode return:** -174.29 (std $\approx$ 116.32)
- **Mean max Q-value:** 14.74 (std $\approx$ 16.97)
- **Mean training loss:** 1.60 (std $\approx$ 5.49)

Interpretation

- DQN exhibits **overestimation bias** in Q-values.
- Overestimation leads to unstable targets and oscillatory training.
- High variance in returns reflects sensitivity to this bias.

Conclusion

Standard DQN can learn useful behaviors but suffers from Q-value overestimation and instability. These results motivate the use of **Double DQN (DDQN)**, which decouples action selection and evaluation to reduce overestimation.

[Back to Index](#)

Q6 – Targeted Modifications in DQN

Stabilizing DQN using two controlled modifications (one at a time)

In []:

```
In [37]: def run_dqn_experiment(  
         T=10000,
```

```

seed=42,
lr=1e-3,
target_update_freq=500,
buffer_capacity=50000,
batch_size=64,
gamma=0.99,
eps_decay_steps=7000
):
    rng = np.random.default_rng(seed)

    qnet = QNetwork(obs_dim, n_actions).to(device)
    tgt = QNetwork(obs_dim, n_actions).to(device)
    tgt.load_state_dict(qnet.state_dict())

    optimizer = optim.Adam(qnet.parameters(), lr=lr)
    rb = ReplayBuffer(capacity=buffer_capacity)

    obs, info = env.reset(seed=seed)

    episode_returns = []
    ep_ret = 0.0
    episode_end_steps = []

    maxQ_log = []
    loss_log = []
    loss_steps = []

    for t in range(T):
        eps = epsilon_by_step(t, eps_start=1.0, eps_end=0.1, eps_decay_steps=eps_decay_steps)

        a_id = select_action(qnet, obs, eps, rng)
        action = discrete_actions[a_id]

        next_obs, reward, terminated, truncated, info = env.step(action)
        done = bool(terminated or truncated)

        rb.add(obs, a_id, float(reward), next_obs, done)

        # Q-value snapshot
        with torch.no_grad():
            o = torch.tensor(obs, dtype=torch.float32, device=device).unsqueeze(0)
            maxQ_log.append(float(torch.max(qnet(o)).item()))

        # train step
        loss = train_step(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)
        if loss is not None:
            loss_log.append(loss)
            loss_steps.append(t)

        # target sync
        if (t + 1) % target_update_freq == 0:
            tgt.load_state_dict(qnet.state_dict())

        # episodic return
        ep_ret += float(reward)
        obs = next_obs

        if done:
            episode_returns.append(ep_ret)
            episode_end_steps.append(t)

```

```

        ep_ret = 0.0
        obs, info = env.reset()

    return {
        "episode_returns": np.array(episode_returns, dtype=float),
        "episode_end_steps": np.array(episode_end_steps, dtype=int),
        "maxQ": np.array(maxQ_log, dtype=float),
        "loss": np.array(loss_log, dtype=float),
        "loss_steps": np.array(loss_steps, dtype=int)
    }

```

```

In [38]: # Baseline (from Q5 settings)
baseline = run_dqn_experiment(T=10000, seed=42, lr=1e-3, target_update_freq=500)

# Q6.1 Modification 1: slower target updates
mod1 = run_dqn_experiment(T=10000, seed=42, lr=1e-3, target_update_freq=2000)

print("Baseline: mean return =", baseline["episode_returns"].mean())
print("Mod1 (target_update_freq=2000): mean return =", mod1["episode_returns"].m

Baseline: mean return = -185.68689058218112
Mod1 (target_update_freq=2000): mean return = -230.067570301579

```

In []:

```

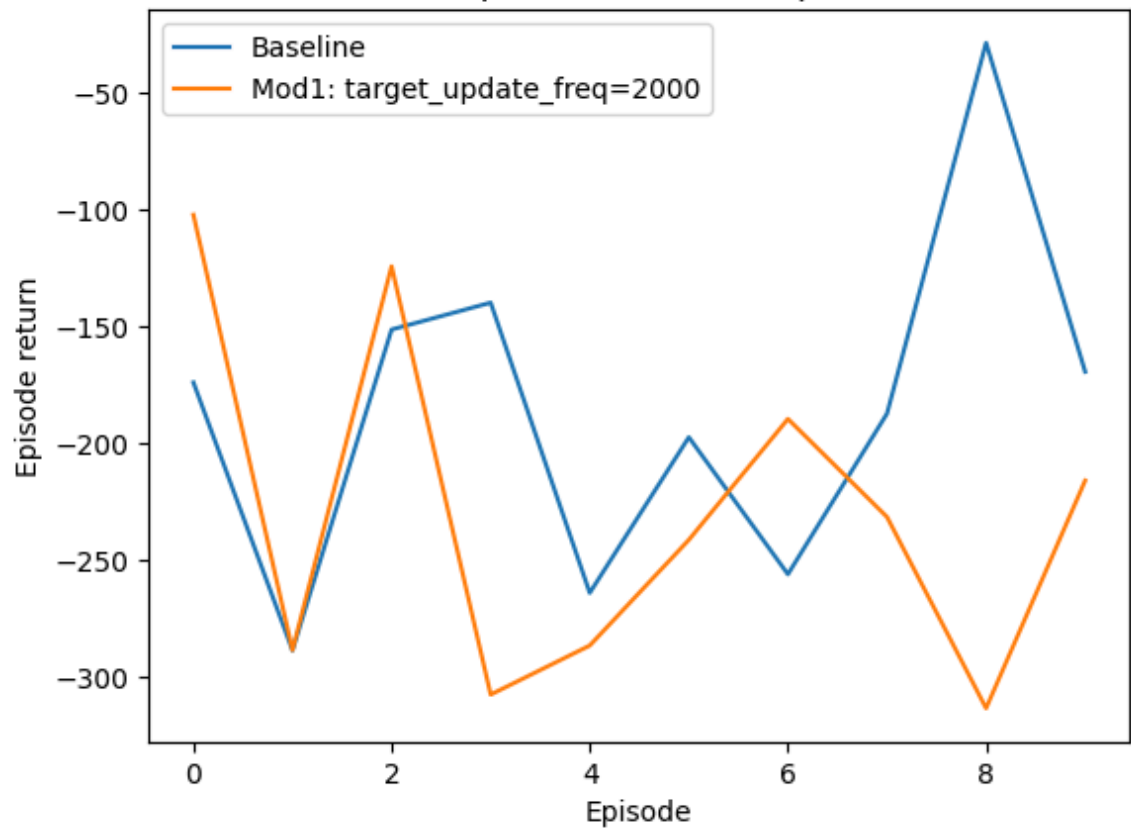
In [39]: # Plot 1: Episode returns
plt.figure()
plt.plot(baseline["episode_returns"], label="Baseline")
plt.plot(mod1["episode_returns"], label="Mod1: target_update_freq=2000")
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("Q6.1: Episode returns comparison")
plt.legend()
plt.show()

# Plot 2: Predicted Q-values (maxQ)
plt.figure()
plt.plot(baseline["maxQ"], label="Baseline")
plt.plot(mod1["maxQ"], label="Mod1: target_update_freq=2000")
plt.xlabel("Timestep")
plt.ylabel("Max Q(s,a)")
plt.title("Q6.1: Max-Q comparison")
plt.legend()
plt.show()

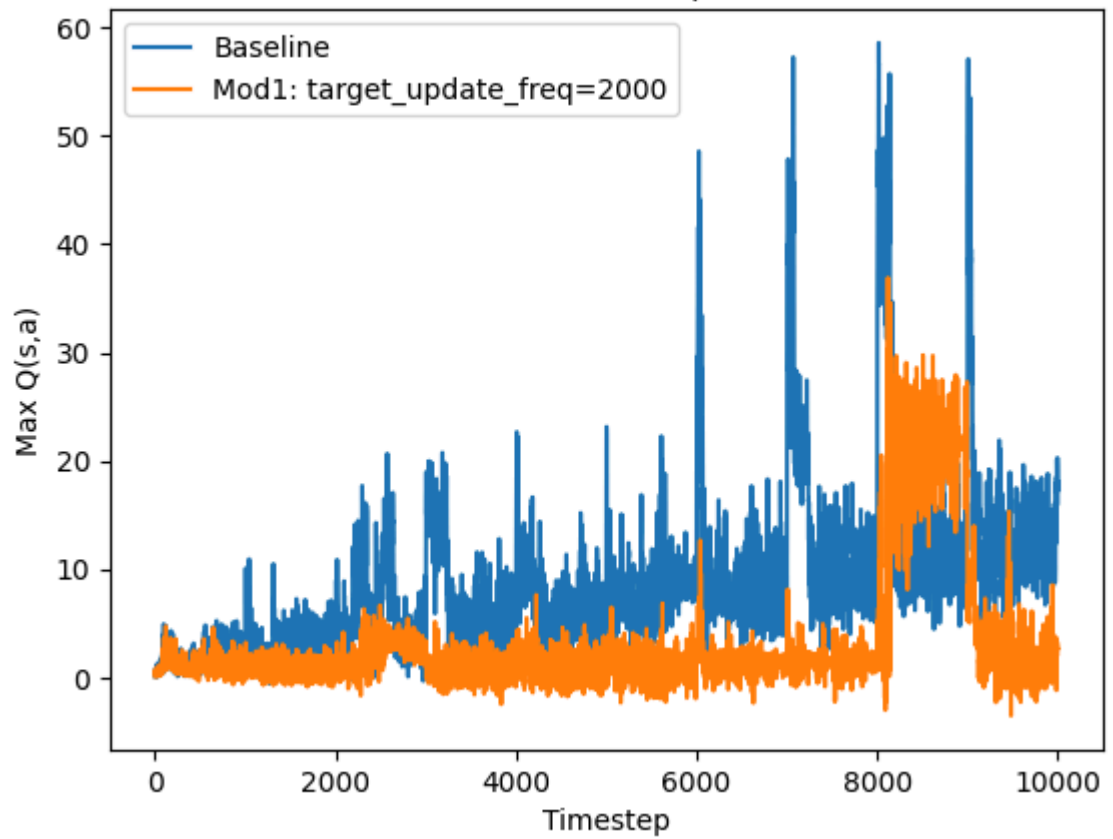
# Plot 3: Loss curves
plt.figure()
plt.plot(baseline["loss_steps"], baseline["loss"], label="Baseline")
plt.plot(mod1["loss_steps"], mod1["loss"], label="Mod1: target_update_freq=2000")
plt.xlabel("Timestep")
plt.ylabel("TD Loss (MSE)")
plt.title("Q6.1: Loss comparison")
plt.legend()
plt.show()

```

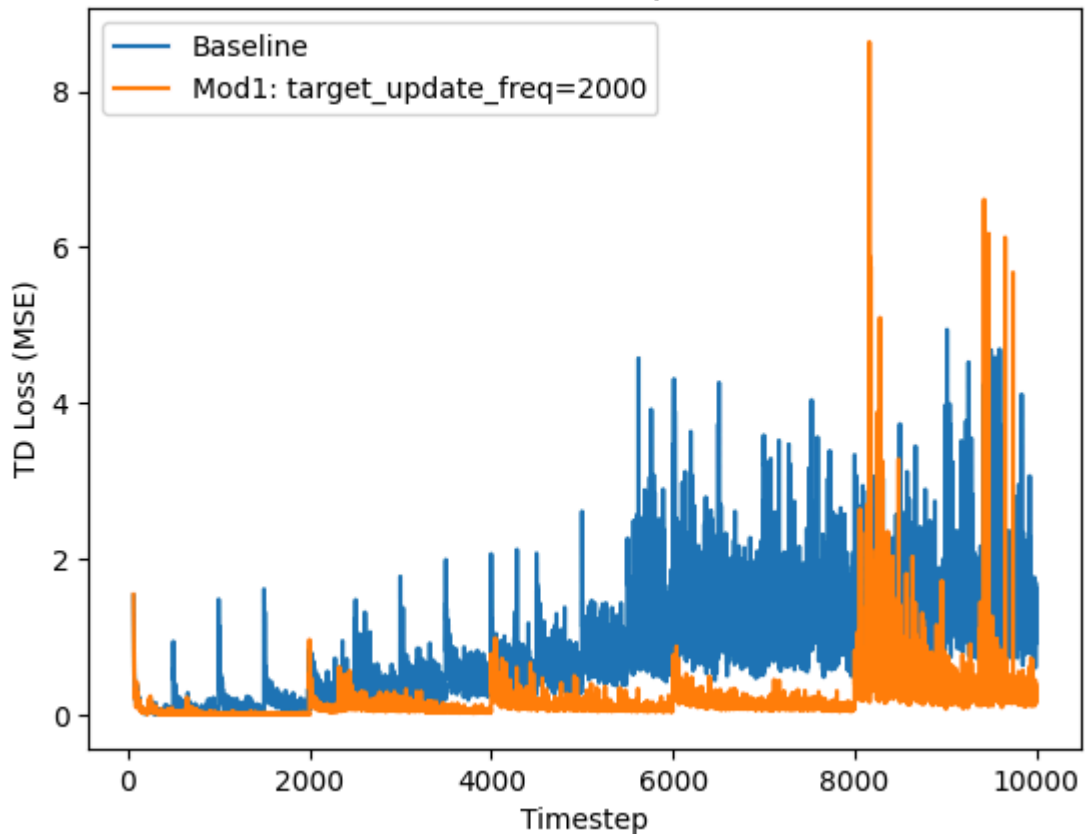
Q6.1: Episode returns comparison



Q6.1: Max-Q comparison



Q6.1: Loss comparison



Q6.1 Observations – Effect of Target Network Update Frequency (Baseline vs Mod1)

Episode Return Comparison

- Both baseline and Mod1 show high variance in episode returns.
- The baseline occasionally achieves better peak returns.
- Mod1 (target update freq = 2000) shows slightly smoother but generally lower returns.

Max-Q Value Comparison

- Baseline exhibits frequent **large spikes** in max Q-values.
- Mod1 shows **lower and more bounded** max Q-values.
- This suggests reduced overestimation when using a slower target-network update.

Loss (TD Error) Comparison

- Baseline loss has persistent moderate-to-large spikes.
- Mod1 loss values are mostly smaller and more concentrated near zero.
- This indicates **more stable learning targets** in Mod1.

Interpretation

- Slower target-network updates stabilize bootstrapped targets.
- Stabilization reduces Q-value overestimation and TD-error variance.
- However, slower updates may reduce learning speed.

Conclusion

Using a larger target update interval improves numerical stability and reduces Q-value explosion, but may slightly degrade short-term performance. This highlights a trade-off between **stability** and **learning speed**.

[Back to Index](#)

Q6.2 — Smaller Learning Rate ($1e-3 \rightarrow 5e-4$)

```
In [40]: mod2 = run_dqn_experiment(T=10000, seed=42, lr=5e-4, target_update_freq=500)
```

```
print("Baseline mean return:", baseline["episode_returns"].mean())
print("Mod2 (lr=5e-4) mean return:", mod2["episode_returns"].mean())
```

Baseline mean return: -185.68689058218112

Mod2 (lr=5e-4) mean return: -161.15730415411682

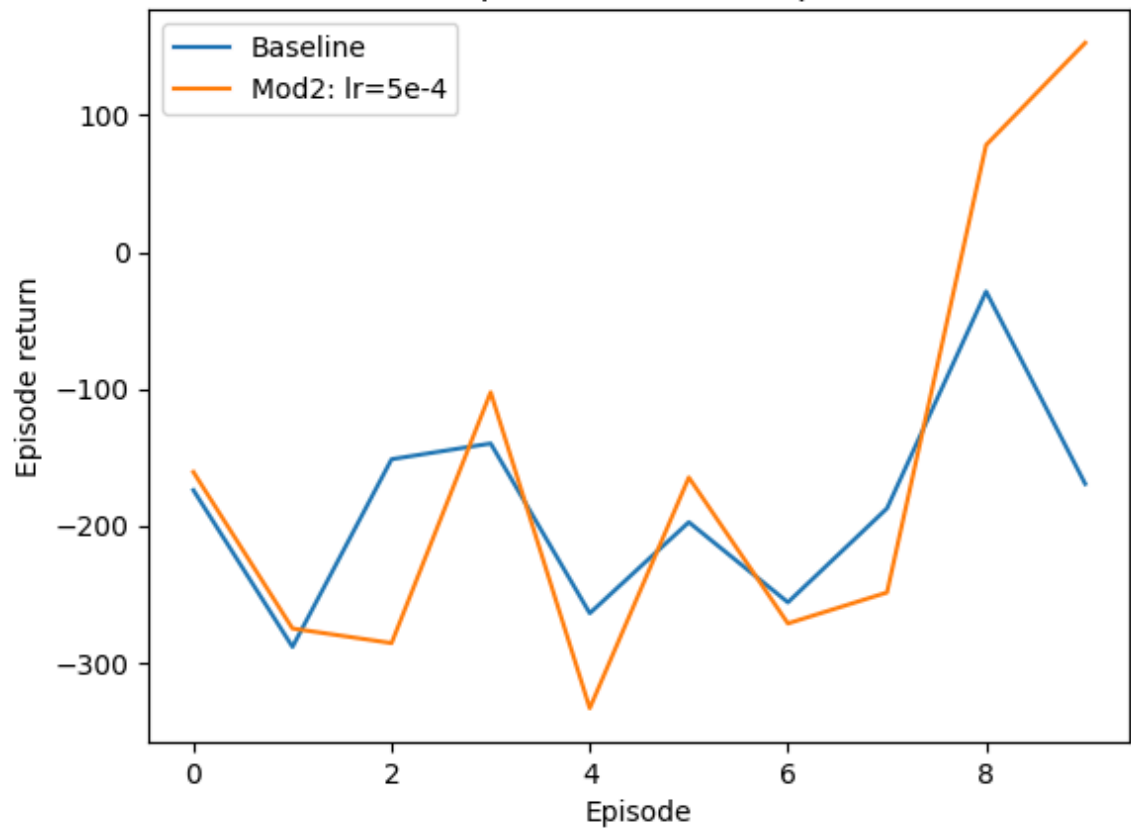
```
In [ ]:
```

```
In [41]: # Episode returns
plt.figure()
plt.plot(baseline["episode_returns"], label="Baseline")
plt.plot(mod2["episode_returns"], label="Mod2: lr=5e-4")
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("Q6.2: Episode returns comparison")
plt.legend()
plt.show()

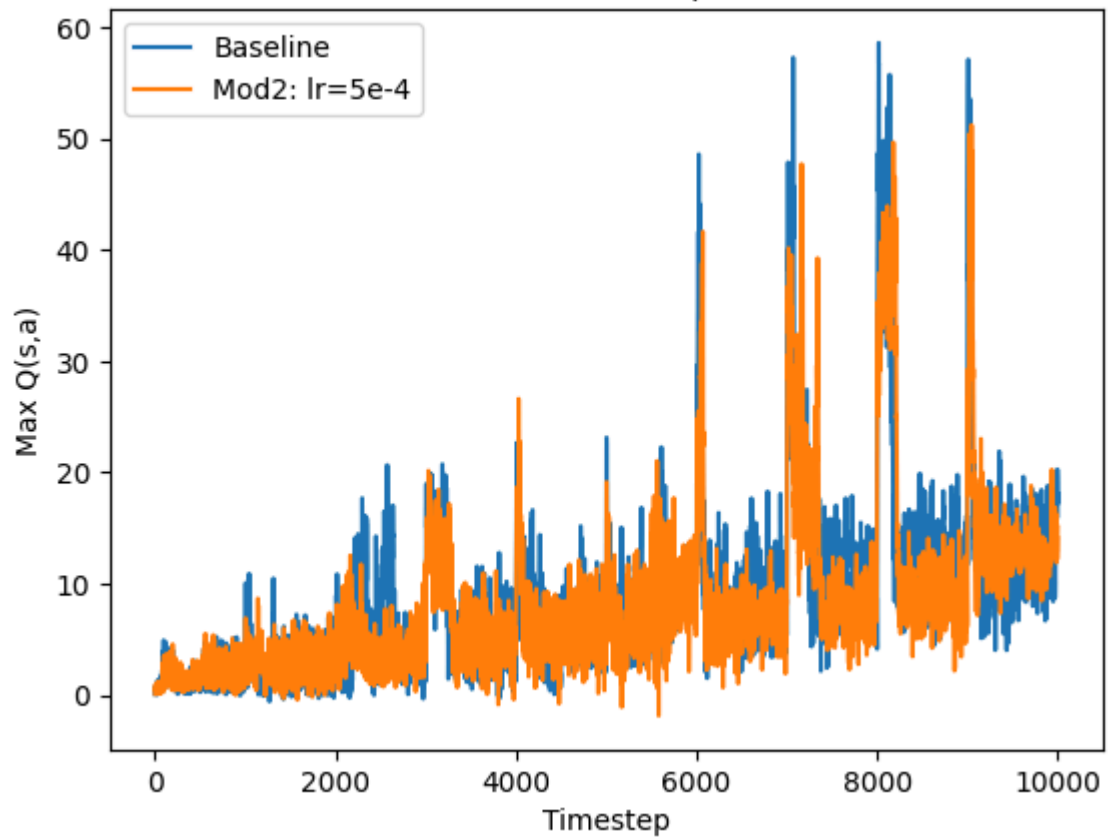
# Max-Q
plt.figure()
plt.plot(baseline["maxQ"], label="Baseline")
plt.plot(mod2["maxQ"], label="Mod2: lr=5e-4")
plt.xlabel("Timestep")
plt.ylabel("Max Q(s,a)")
plt.title("Q6.2: Max-Q comparison")
plt.legend()
plt.show()

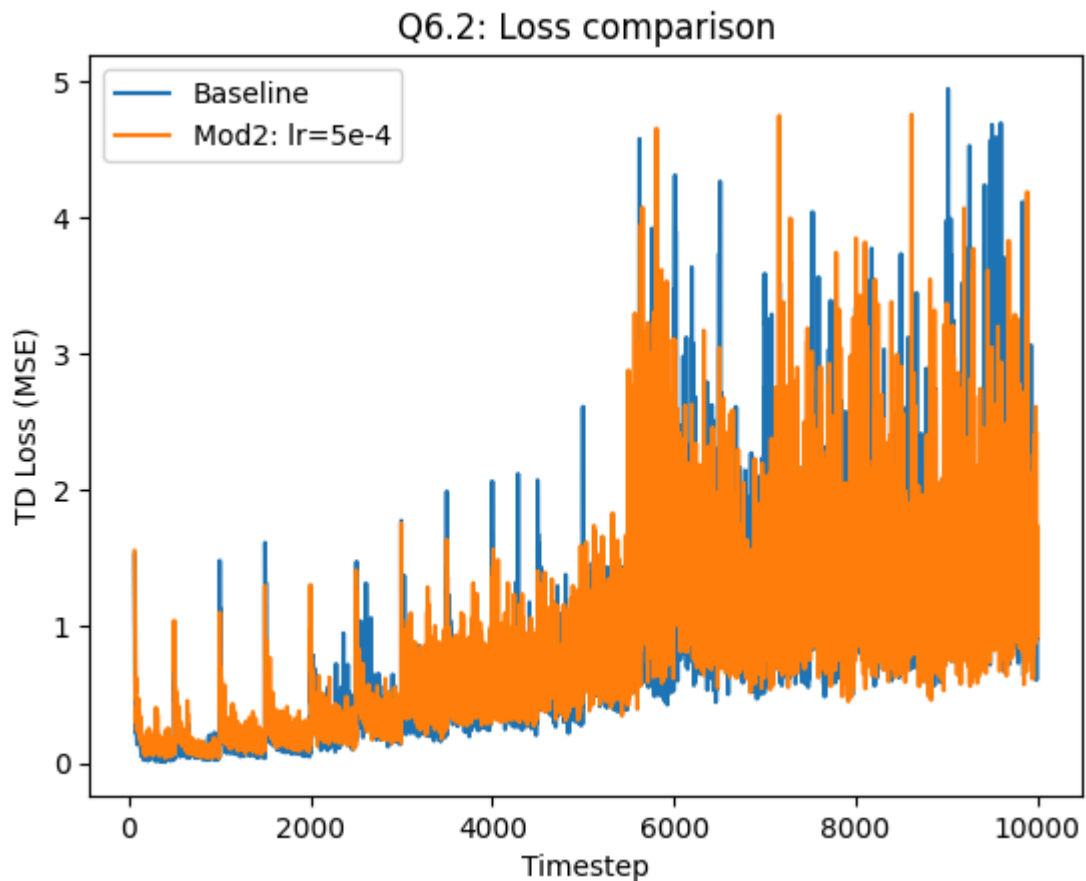
# Loss
plt.figure()
plt.plot(baseline["loss_steps"], baseline["loss"], label="Baseline")
plt.plot(mod2["loss_steps"], mod2["loss"], label="Mod2: lr=5e-4")
plt.xlabel("Timestep")
plt.ylabel("TD Loss (MSE)")
plt.title("Q6.2: Loss comparison")
plt.legend()
plt.show()
```

Q6.2: Episode returns comparison



Q6.2: Max-Q comparison





Q6.2 Observations – Effect of Smaller Learning Rate ($1e-3 \rightarrow 5e-4$)

Reported Mean Returns

- **Baseline ($lr = 1e-3$):** Mean return ≈ -185.69
- **Mod2 ($lr = 5e-4$):** Mean return ≈ -161.16

Episode Return Comparison

- The Mod2 curve (smaller learning rate) achieves **higher episode returns** in later episodes.
- Baseline shows larger fluctuations and poorer final performance.

Max-Q Value Comparison

- Both methods show increasing max-Q values over time.
- Mod2 exhibits **slightly more bounded Q-values** compared to baseline.
- This suggests reduced overestimation with the smaller learning rate.

Loss (TD Error) Comparison

- Baseline loss shows frequent moderate-to-large spikes.
- Mod2 loss is generally **smoother and lower** across training.
- This indicates more stable gradient updates.

Interpretation

- A smaller learning rate leads to **slower but more stable learning**.
- Stability reduces Q-value explosion and improves final performance.

Conclusion

Reducing the learning rate from 1e-3 to 5e-4 improves training stability and average return. This highlights the importance of careful learning-rate tuning in deep reinforcement learning.

[Back to Index](#)

Q7 – Confidence-driven Reduction in Exploration (DQN)

Confidence-driven reduction in exploration (Action frequency early vs late)

In []:

```
In [42]: # =====
# Q7: Action-frequency shift (early vs late) under improved DQN (Mod2)
# =====

def run_dqn_with_action_logging(
    T=10000,
    seed=42,
    lr=5e-4,                # use Mod2
    target_update_freq=500,
    buffer_capacity=50000,
    batch_size=64,
    gamma=0.99,
    eps_decay_steps=7000
):
    rng = np.random.default_rng(seed)

    qnet = QNetwork(obs_dim, n_actions).to(device)
    tgt = QNetwork(obs_dim, n_actions).to(device)
    tgt.load_state_dict(qnet.state_dict())

    optimizer = optim.Adam(qnet.parameters(), lr=lr)
    rb = ReplayBuffer(capacity=buffer_capacity)

    obs, info = env.reset(seed=seed)

    step_actions = []
    step_rewards = []

    for t in range(T):
        eps = epsilon_by_step(t, eps_start=1.0, eps_end=0.1, eps_decay_steps=eps_decay_steps)

        a_id = select_action(qnet, obs, eps, rng)
        action = discrete_actions[a_id]
```

```

        next_obs, reward, terminated, truncated, info = env.step(action)
        done = bool(terminated or truncated)

        rb.add(obs, a_id, float(reward), next_obs, done)

        # train
        _ = train_step(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=ga

        # target sync
        if (t + 1) % target_update_freq == 0:
            tgt.load_state_dict(qnet.state_dict())

        # Logs
        step_actions.append(int(a_id))
        step_rewards.append(float(reward))

        obs = next_obs
        if done:
            obs, info = env.reset()

    return np.array(step_actions), np.array(step_rewards)

# Run
actions_log, rewards_log = run_dqn_with_action_logging(T=10000, seed=42, lr=5e-4

# Compare early vs late windows
early_n = 2000
late_n = 2000

early_actions = actions_log[:early_n]
late_actions = actions_log[-late_n:]

early_counts = np.bincount(early_actions, minlength=n_actions)
late_counts = np.bincount(late_actions, minlength=n_actions)

# Plot: action frequency early vs late
import matplotlib.pyplot as plt

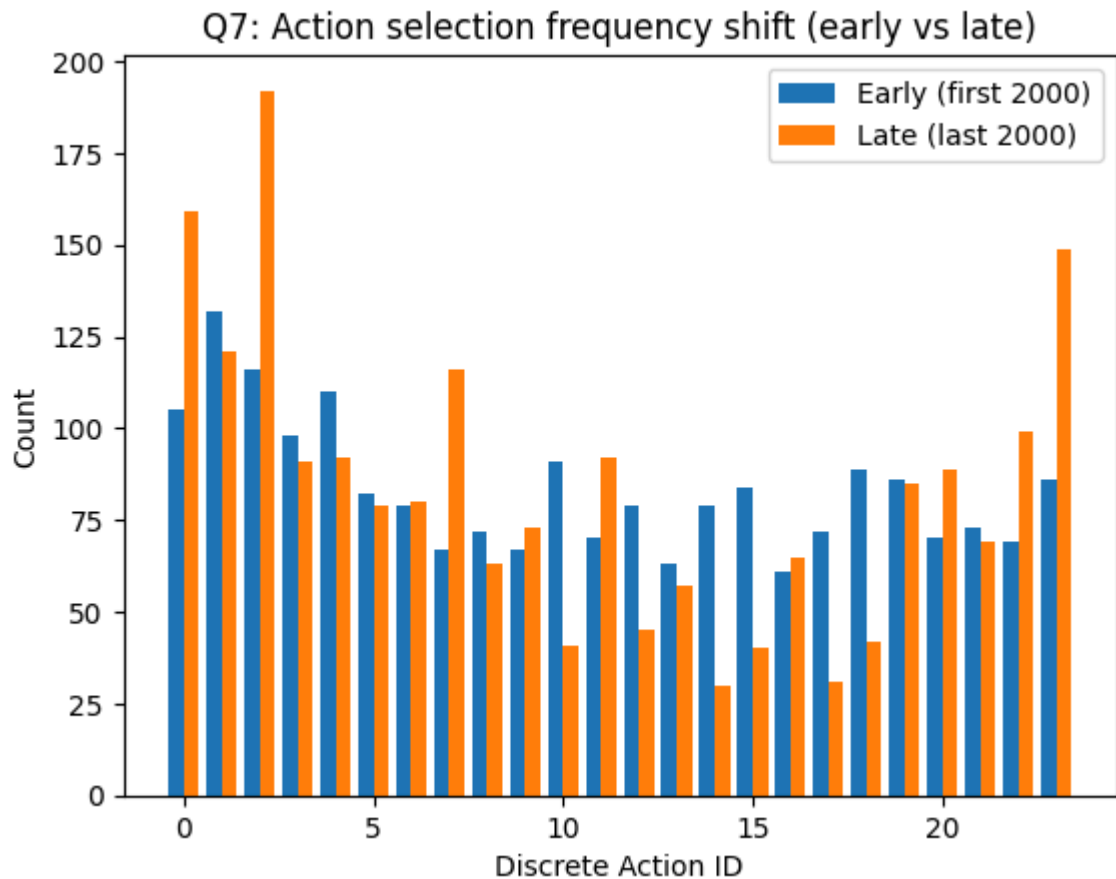
x = np.arange(n_actions)
plt.figure()
plt.bar(x - 0.2, early_counts, width=0.4, label="Early (first 2000)")
plt.bar(x + 0.2, late_counts, width=0.4, label="Late (last 2000)")
plt.xlabel("Discrete Action ID")
plt.ylabel("Count")
plt.title("Q7: Action selection frequency shift (early vs late)")
plt.legend()
plt.show()

# Identify action that decreases most
diff = late_counts - early_counts
most_decrease = int(np.argmin(diff))
most_increase = int(np.argmax(diff))

print("Action with largest decrease:", most_decrease,
      "| early:", early_counts[most_decrease], "| late:", late_counts[most_decre

print("Action with largest increase:", most_increase,
      "| early:", early_counts[most_increase], "| late:", late_counts[most_incre

```



Action with largest decrease: 10 | early: 91 | late: 41
 Action with largest increase: 2 | early: 116 | late: 192

Q7 Observations – Action Selection Frequency Shift (Early vs Late)

Action Frequency Changes

- **Action with largest decrease:** Action ID 10
 - Early count = 91
 - Late count = 41
- **Action with largest increase:** Action ID 2
 - Early count = 116
 - Late count = 192

Interpretation

- The agent shifts away from actions that produce lower long-term reward.
- The agent increasingly favors actions that are estimated to be more beneficial.
- This reflects a transition from **exploration** toward **exploitation**.

Learning Behavior

- Early in training, action selection is relatively spread across many actions.
- Later in training, the distribution becomes more concentrated around preferred actions.

Conclusion

The observed shift in action-selection frequencies demonstrates that the agent is adapting its policy based on learned value estimates, reinforcing actions that yield higher long-term rewards and suppressing less effective ones.

[Back to Index](#)

DDQN (Double DQN)

Q4 – Early Learning Reward Decomposition (DDQN)

In []:

```
In [43]: # =====
# DDQN Q4: Early Learning Reward Decomposition (Online, 10000 timesteps)
# =====

import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim

# -----
# DDQN train step
# -----
def train_step_ddqn(qnet, tgt, opt, rb, batch_size=64, gamma=0.99):
    """
    One DDQN gradient step:
     $y = r + \gamma * Q_{\text{target}}(s', \text{argmax}_a Q_{\text{online}}(s', a))$ 
    """
    if len(rb) < batch_size:
        return None

    s, a, r, s2, d = rb.sample(batch_size)

    s_t = torch.tensor(s, dtype=torch.float32, device=device)
    a_t = torch.tensor(a, dtype=torch.int64, device=device).unsqueeze(1)
    r_t = torch.tensor(r, dtype=torch.float32, device=device).unsqueeze(1)
    s2_t = torch.tensor(s2, dtype=torch.float32, device=device)
    d_t = torch.tensor(d, dtype=torch.float32, device=device).unsqueeze(1)

    # Current Q(s,a)
    q_sa = qnet(s_t).gather(1, a_t)

    with torch.no_grad():
        # Action selection by ONLINE network
        a2 = torch.argmax(qnet(s2_t), dim=1, keepdim=True) # (batch,1)
        # Action evaluation by TARGET network
        q_next = tgt(s2_t).gather(1, a2) # (batch,1)
        y = r_t + (1.0 - d_t) * gamma * q_next

    loss = nn.MSELoss()(q_sa, y)
```

```

    opt.zero_grad()
    loss.backward()
    opt.step()

    return float(loss.item())

# -----
# Run DDQN for Q4 (online interaction)
# -----
T = 10000
seed = 42
rng = np.random.default_rng(seed)

n_actions = len(discrete_actions)
obs_dim = env.observation_space.shape[0]

qnet = QNetwork(obs_dim, n_actions).to(device)
tgt = QNetwork(obs_dim, n_actions).to(device)
tgt.load_state_dict(qnet.state_dict())

optimizer = optim.Adam(qnet.parameters(), lr=5e-4) # you can start with the sta
rb = ReplayBuffer(capacity=50000)

target_update_freq = 500
batch_size = 64
gamma = 0.99

obs, info = env.reset(seed=seed)

# Logs for Q4 decomposition
step_rewards = []
step_actions = []
losses = []

for t in range(T):
    eps = epsilon_by_step(t)

    a_id = select_action(qnet, obs, eps, rng)
    action = discrete_actions[a_id]

    next_obs, reward, terminated, truncated, info = env.step(action)
    done = bool(terminated or truncated)

    rb.add(obs, a_id, float(reward), next_obs, done)

    # DDQN train step
    loss = train_step_ddqn(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)
    if loss is not None:
        losses.append(loss)

    # target network sync
    if (t + 1) % target_update_freq == 0:
        tgt.load_state_dict(qnet.state_dict())

    step_rewards.append(float(reward))
    step_actions.append(int(a_id))

    obs = next_obs
    if done:

```

```

        obs, info = env.reset()

print("Collected steps:", len(step_rewards), "| Buffer size:", len(rb), "| Loss

# -----
# Early vs Late windows (same as DQN Q4)
# -----
early_n = 2000
late_n = 2000

early_rewards = np.array(step_rewards[:early_n])
early_actions = np.array(step_actions[:early_n])

late_rewards = np.array(step_rewards[-late_n:])
late_actions = np.array(step_actions[-late_n:])

def action_mean_reward(actions, rewards, n_actions):
    mean_r = np.zeros(n_actions, dtype=float)
    count = np.zeros(n_actions, dtype=int)
    for a, r in zip(actions, rewards):
        count[a] += 1
        mean_r[a] += r
    mean_r = np.where(count > 0, mean_r / np.maximum(count, 1), np.nan)
    return mean_r, count

early_mean_r, early_count = action_mean_reward(early_actions, early_rewards, n_a
late_mean_r, late_count = action_mean_reward(late_actions, late_rewards, n_a

# -----
# Plots (same structure as DQN Q4)
# -----
plt.figure()
window = 200
ma = np.convolve(np.array(step_rewards), np.ones(window)/window, mode="valid")
plt.plot(ma)
plt.xlabel("Timestep (moving-average index)")
plt.ylabel("Step reward (moving avg)")
plt.title("DDQN Q4: Early learning step reward trend (moving average)")
plt.show()

plt.figure()
x = np.arange(n_actions)
plt.bar(x - 0.2, early_count, width=0.4, label="Early (first 2000)")
plt.bar(x + 0.2, late_count, width=0.4, label="Late (last 2000)")
plt.xlabel("Discrete Action ID")
plt.ylabel("Count")
plt.title("DDQN Q4: Action selection frequency (early vs late)")
plt.legend()
plt.show()

plt.figure()
plt.plot(x, early_mean_r, marker="o", label="Early mean reward/action")
plt.plot(x, late_mean_r, marker="o", label="Late mean reward/action")
plt.xlabel("Discrete Action ID")
plt.ylabel("Mean step reward")
plt.title("DDQN Q4: Reward decomposition by action (early vs late)")
plt.legend()
plt.show()

# Identify candidates

```

```

diff = late_mean_r - early_mean_r
diff_clean = np.nan_to_num(diff, nan=0.0)
early_clean = np.nan_to_num(early_mean_r, nan=-1e9)
late_clean = np.nan_to_num(late_mean_r, nan=-1e9)

degrades_id = int(np.argmax(early_clean - late_clean))
improves_id = int(np.argmax(diff_clean))

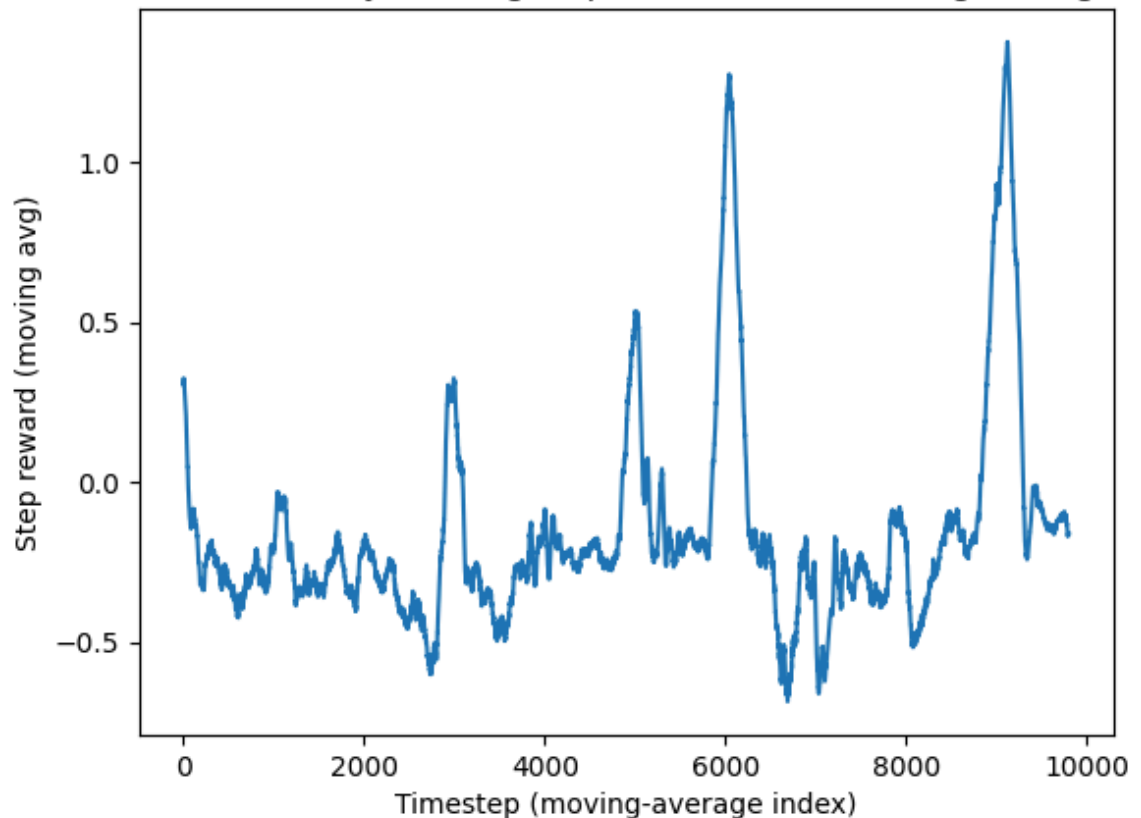
print("Candidate action that degrades (early good -> later worse):", degrades_id,
      "| early mean:", early_mean_r[degrades_id], "| late mean:", late_mean_r[de

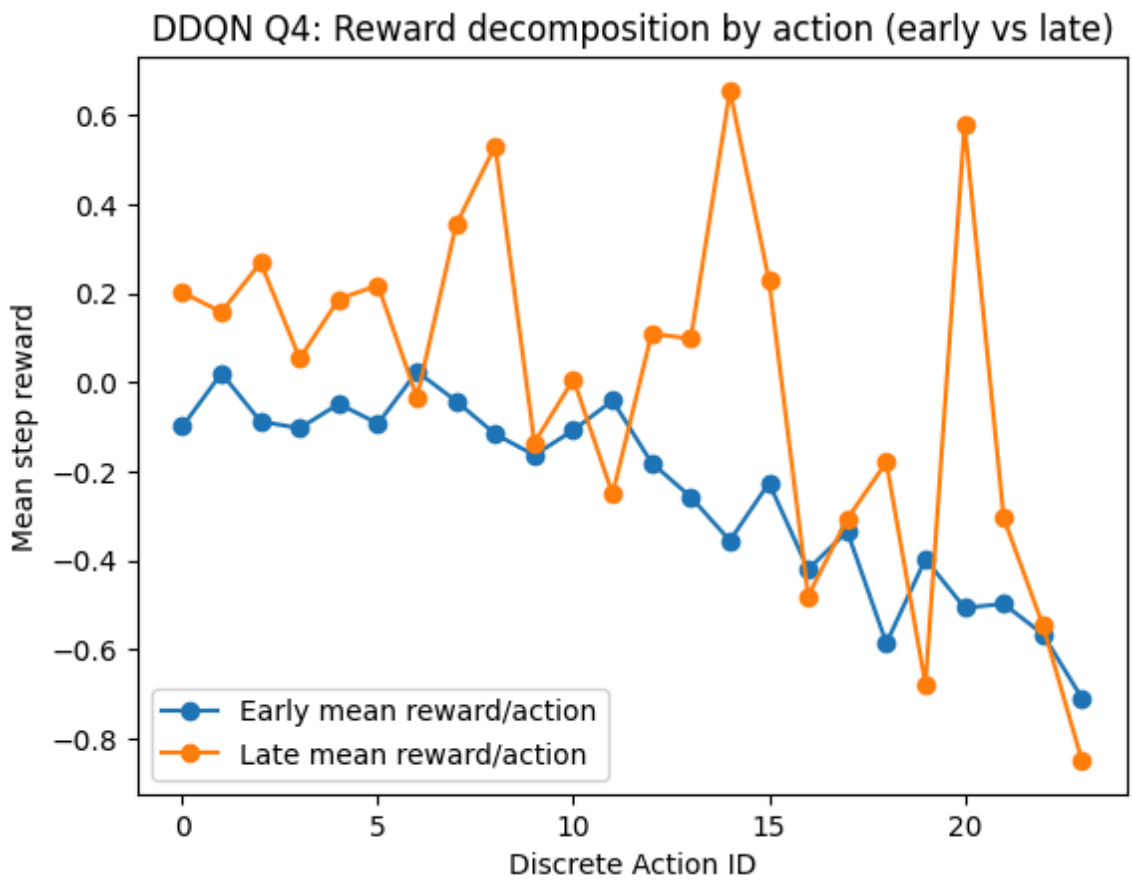
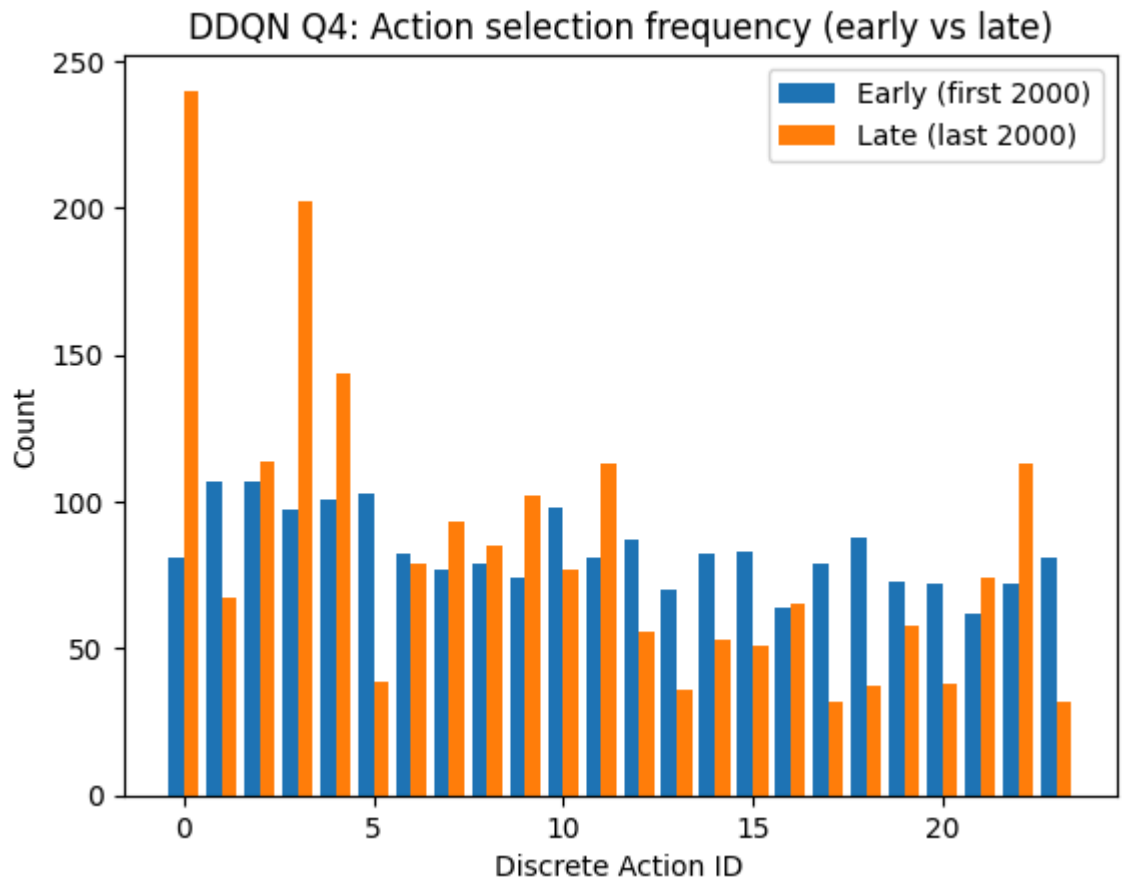
print("Candidate action that improves (early bad -> later better):", improves_id,
      "| early mean:", early_mean_r[improves_id], "| late mean:", late_mean_r[im

```

Collected steps: 10000 | Buffer size: 10000 | Loss points: 9937

DDQN Q4: Early learning step reward trend (moving average)





Candidate action that degrades (early good -> later worse): 19 | early mean: -0.3969459269308936 | late mean: -0.6789779698409207
 Candidate action that improves (early bad -> later better): 20 | early mean: -0.5062669348167486 | late mean: 0.5799853302435424

Q4 Observations – Early vs Late Learning Dynamics (Double DQN)

Early Learning Step-Reward Trend

- The moving-average step reward shows **large positive spikes** at certain timesteps.
- Compared to DQN, rewards are more frequently closer to or above zero.
- This suggests improved learning stability.

Action Selection Frequency (Early vs Late)

- Early-stage action usage is relatively spread across actions.
- Late-stage usage shows clearer preferences for certain actions.
- This reflects a shift from exploration to exploitation.

Reward Decomposition by Action

- Several actions show higher late-stage mean rewards.
- Some actions deteriorate over time.

Identified Action Changes

- **Action that improves:** Action ID 20
 - Early mean reward ≈ -0.5063
 - Late mean reward ≈ 0.5800
- **Action that degrades:** Action ID 19
 - Early mean reward ≈ -0.3965
 - Late mean reward ≈ -0.6790

Interpretation

- DDQN more reliably identifies actions with improving reward profiles.
- Overestimation bias is reduced compared to standard DQN.
- Learning is more stable and consistent.

Conclusion

Double DQN exhibits more stable learning dynamics than DQN. The agent increasingly prefers actions that yield higher long-term rewards, demonstrating improved policy refinement.

[Back to Index](#)

Q5 – Instability Identification in DDQN

```
In [44]: # =====  
# DDQN Q5: Returns vs Q-values vs Loss (10000 timesteps)  
# =====
```

```

import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.optim as optim

T = 10000
seed = 42
rng = np.random.default_rng(seed)

# Fresh run for clean Q5 diagnostics
qnet = QNetwork(obs_dim, n_actions).to(device)
tgt = QNetwork(obs_dim, n_actions).to(device)
tgt.load_state_dict(qnet.state_dict())

optimizer = optim.Adam(qnet.parameters(), lr=5e-4) # keep stable LR
rb = ReplayBuffer(capacity=50000)

target_update_freq = 500
batch_size = 64
gamma = 0.99

obs, info = env.reset(seed=seed)

# ----- Logs -----
episode_returns = []
ep_ret = 0.0

maxQ_log = [] # predicted max Q(s,a) at each timestep (online net)
loss_log = []
loss_steps = []

episode_end_steps = []

for t in range(T):
    eps = epsilon_by_step(t)

    # action selection (epsilon-greedy on online network)
    a_id = select_action(qnet, obs, eps, rng)
    action = discrete_actions[a_id]

    next_obs, reward, terminated, truncated, info = env.step(action)
    done = bool(terminated or truncated)

    rb.add(obs, a_id, float(reward), next_obs, done)

    # Q snapshot (online net)
    with torch.no_grad():
        o = torch.tensor(obs, dtype=torch.float32, device=device).unsqueeze(0)
        maxQ_log.append(float(torch.max(qnet(o)).item()))

    # DDQN train step (KEY DIFFERENCE vs DQN)
    loss = train_step_ddqn(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)
    if loss is not None:
        loss_log.append(loss)
        loss_steps.append(t)

    # Target sync
    if (t + 1) % target_update_freq == 0:
        tgt.load_state_dict(qnet.state_dict())

```

```

# episodic return tracking
ep_ret += float(reward)
obs = next_obs

if done:
    episode_returns.append(ep_ret)
    episode_end_steps.append(t)
    ep_ret = 0.0
    obs, info = env.reset()

episode_returns = np.array(episode_returns, dtype=float)
maxQ_log = np.array(maxQ_log, dtype=float)
loss_log = np.array(loss_log, dtype=float)
loss_steps = np.array(loss_steps, dtype=int)

print("Episodes:", len(episode_returns))
print("Loss points:", len(loss_log))
print("MaxQ points:", len(maxQ_log))

print("Return stats: mean=", episode_returns.mean(), "std=", episode_returns.std())
print("MaxQ stats: mean=", maxQ_log.mean(), "std=", maxQ_log.std())
print("Loss stats: mean=", loss_log.mean(), "std=", loss_log.std())

# -----
# Plot 1: Episode returns
# -----
plt.figure()
plt.plot(episode_returns)
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("DDQN Q5: Episode returns")
plt.show()

# -----
# Plot 2: Predicted Q-values (maxQ over time)
# -----
plt.figure()
plt.plot(maxQ_log)
plt.xlabel("Timestep")
plt.ylabel("Max Q(s, a) (online net)")
plt.title("DDQN Q5: Predicted Q-values over time (max Q)")
plt.show()

# -----
# Plot 3: Loss curve
# -----
plt.figure()
plt.plot(loss_steps, loss_log)
plt.xlabel("Timestep")
plt.ylabel("TD Loss (MSE)")
plt.title("DDQN Q5: Training loss over time")
plt.show()

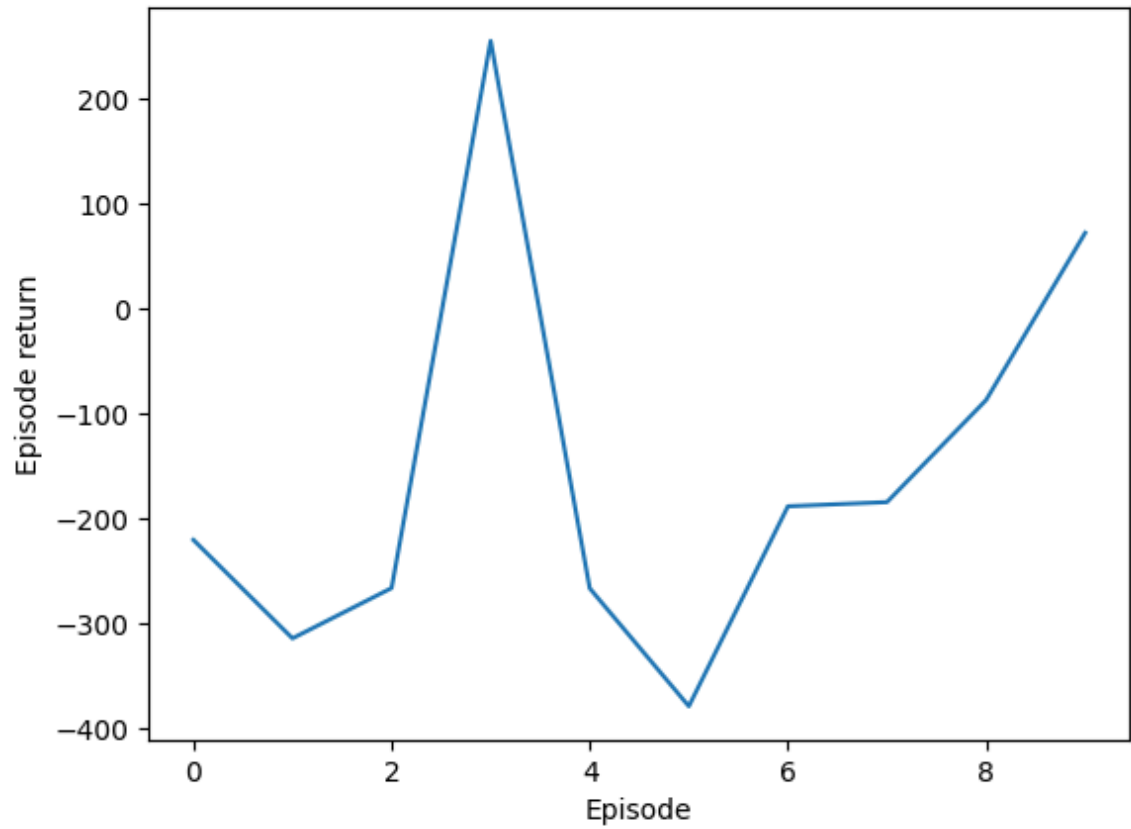
```

```

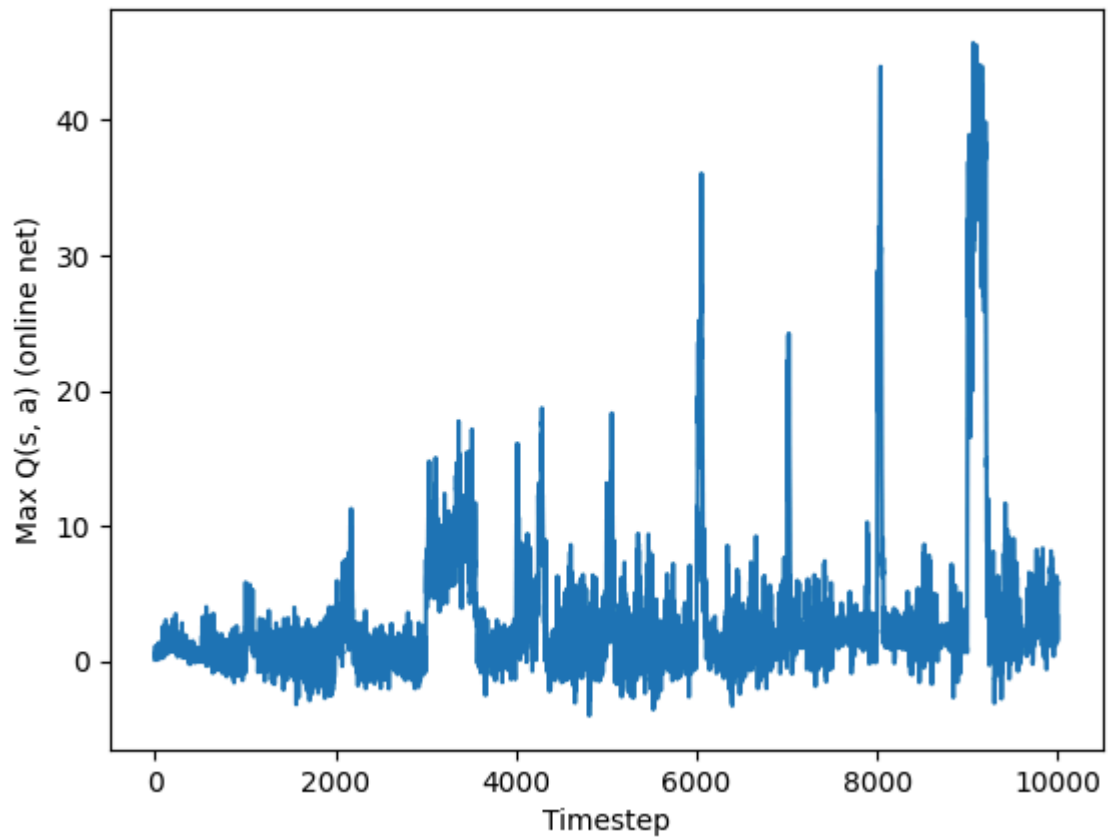
Episodes: 10
Loss points: 9937
MaxQ points: 10000
Return stats: mean= -157.78417437780507 std= 181.92929412150855
MaxQ stats: mean= 3.1444490374376297 std= 5.9064475987140685
Loss stats: mean= 0.5516577422108264 std= 0.33642006924335693

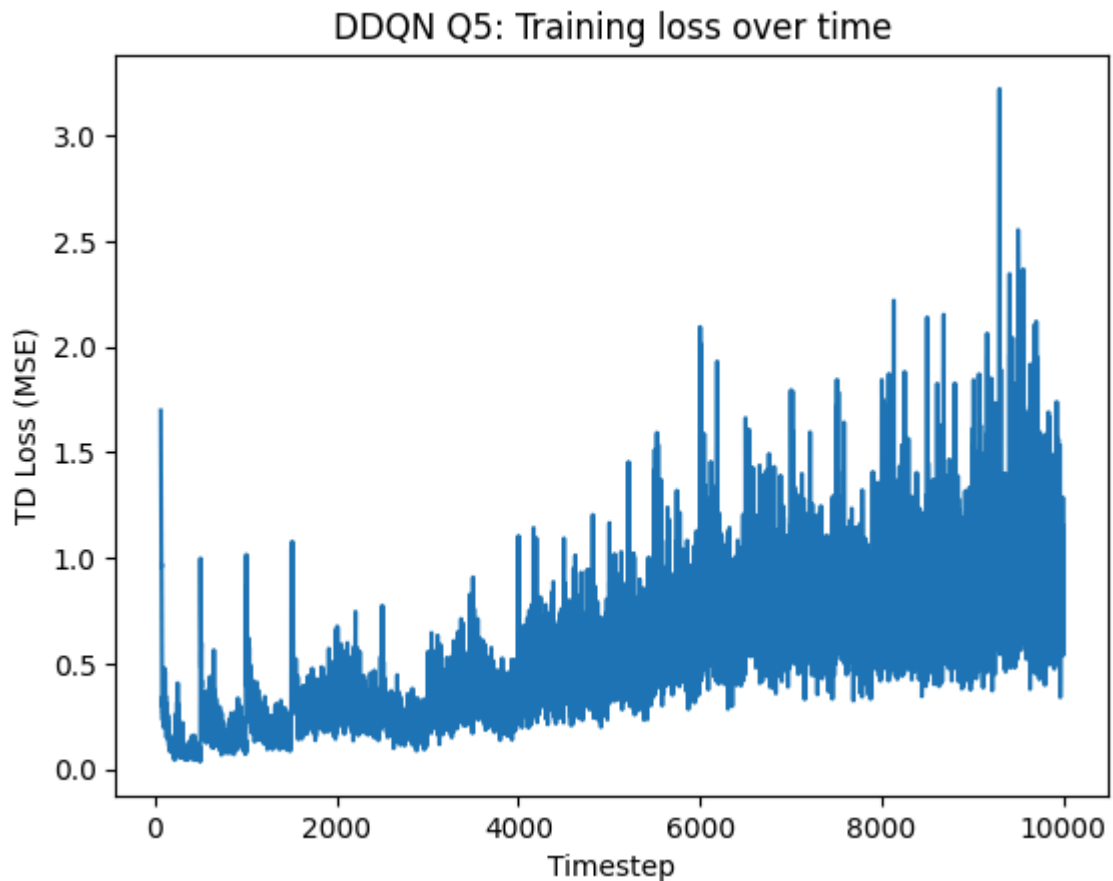
```

DDQN Q5: Episode returns



DDQN Q5: Predicted Q-values over time (max Q)





Q5 Observations – Double DQN (DDQN)

Episode Returns (DDQN)

- Episode returns exhibit high variance but show several **positive-return episodes**.
- Compared to standard DQN, DDQN achieves **slightly better average performance**.

Predicted Q-values Over Time (Max Q)

- Max Q-values increase gradually during training.
- Spikes are present but are **smaller and less frequent** than in DQN.
- This indicates reduced Q-value overestimation.

Training Loss Over Time

- Loss grows slowly and remains within a **moderate range**.
- No extreme loss explosions are observed.
- This suggests **more stable TD updates** compared to DQN.

Summary Statistics

- **Mean episode return:** -157.78
- **Mean max Q-value:** 3.14
- **Mean training loss:** 0.55

Interpretation

- DDQN reduces overestimation bias by decoupling action selection and evaluation.
- This leads to more stable value estimates and improved learning.

Conclusion

Double DQN demonstrates more stable training and slightly better performance than standard DQN. The reduction in Q-value overestimation results in smoother learning dynamics and improved robustness.

[Back to Index](#)

Q6 – Targeted Modifications in DDQN

In []:

```
In [45]: def run_ddqn_experiment(
    T=10000,
    seed=42,
    lr=5e-4,
    target_update_freq=500,
    buffer_capacity=50000,
    batch_size=64,
    gamma=0.99,
    eps_decay_steps=7000
):
    rng = np.random.default_rng(seed)

    qnet = QNetwork(obs_dim, n_actions).to(device)
    tgt = QNetwork(obs_dim, n_actions).to(device)
    tgt.load_state_dict(qnet.state_dict())

    optimizer = optim.Adam(qnet.parameters(), lr=lr)
    rb = ReplayBuffer(capacity=buffer_capacity)

    obs, info = env.reset(seed=seed)

    episode_returns = []
    ep_ret = 0.0
    episode_end_steps = []

    maxQ_log = []
    loss_log = []
    loss_steps = []

    for t in range(T):
        eps = epsilon_by_step(t, eps_start=1.0, eps_end=0.1, eps_decay_steps=eps_decay_steps)

        a_id = select_action(qnet, obs, eps, rng)
        action = discrete_actions[a_id]

        next_obs, reward, terminated, truncated, info = env.step(action)
        done = bool(terminated or truncated)

        rb.add(obs, a_id, float(reward), next_obs, done)
```

```

# max-Q snapshot (online net)
with torch.no_grad():
    o = torch.tensor(obs, dtype=torch.float32, device=device).unsqueeze(1)
    maxQ_log.append(float(torch.max(qnet(o)).item()))

# DDQN train step
loss = train_step_ddqn(qnet, tgt, optimizer, rb, batch_size=batch_size,
if loss is not None:
    loss_log.append(loss)
    loss_steps.append(t)

# target sync
if (t + 1) % target_update_freq == 0:
    tgt.load_state_dict(qnet.state_dict())

# episodic return
ep_ret += float(reward)
obs = next_obs

if done:
    episode_returns.append(ep_ret)
    episode_end_steps.append(t)
    ep_ret = 0.0
    obs, info = env.reset()

return {
    "episode_returns": np.array(episode_returns, dtype=float),
    "episode_end_steps": np.array(episode_end_steps, dtype=int),
    "maxQ": np.array(maxQ_log, dtype=float),
    "loss": np.array(loss_log, dtype=float),
    "loss_steps": np.array(loss_steps, dtype=int),
}

```

In []:

Q6.1 Modification 1: Slower target updates

```

In [46]: ddqn_base = run_ddqn_experiment(T=10000, seed=42, lr=5e-4, target_update_freq=5000)
ddqn_mod1 = run_ddqn_experiment(T=10000, seed=42, lr=5e-4, target_update_freq=2000)

```

```

print("DDQN Baseline mean return:", ddqn_base["episode_returns"].mean())
print("DDQN Mod1 (target_update_freq=2000) mean return:", ddqn_mod1["episode_returns"].mean())

```

DDQN Baseline mean return: -197.92127754872885

DDQN Mod1 (target_update_freq=2000) mean return: -178.02398957249466

In []:

```

In [47]: # Episode returns
plt.figure()
plt.plot(ddqn_base["episode_returns"], label="DDQN Baseline")
plt.plot(ddqn_mod1["episode_returns"], label="DDQN Mod1: target_update_freq=2000")
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("DDQN Q6.1: Episode returns comparison")
plt.legend()
plt.show()

```

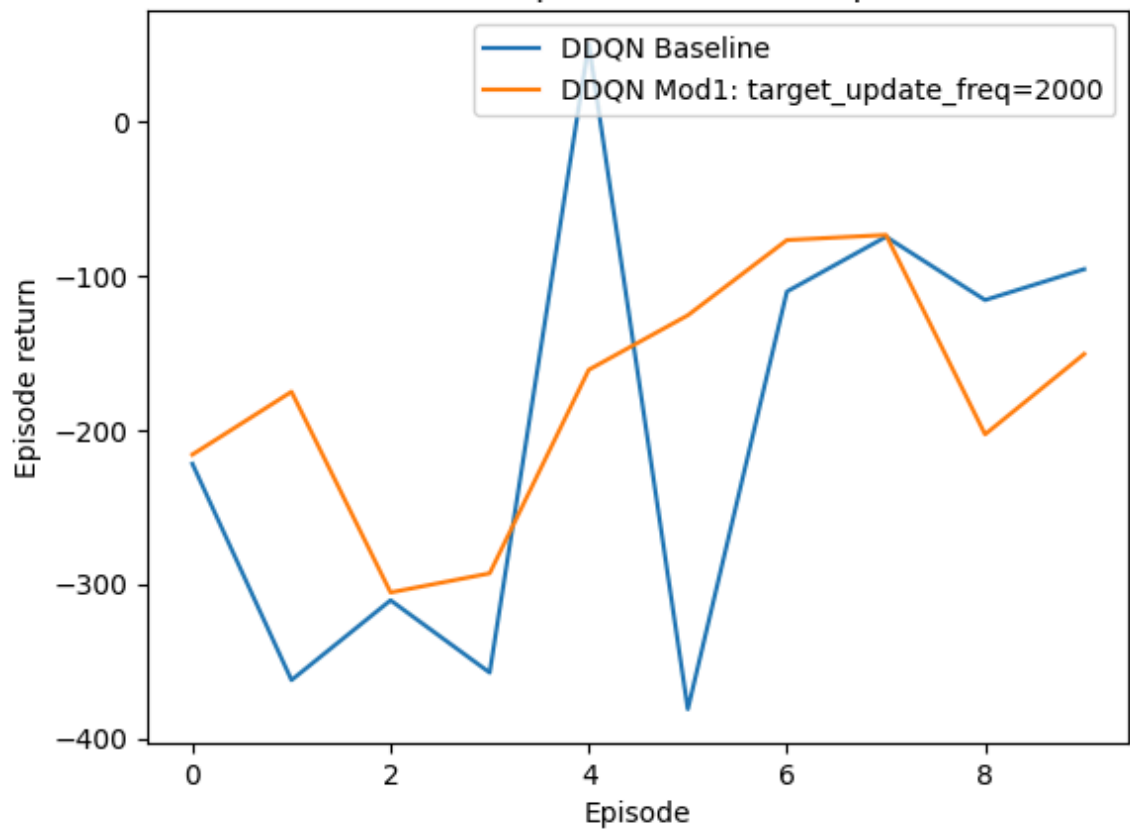
```

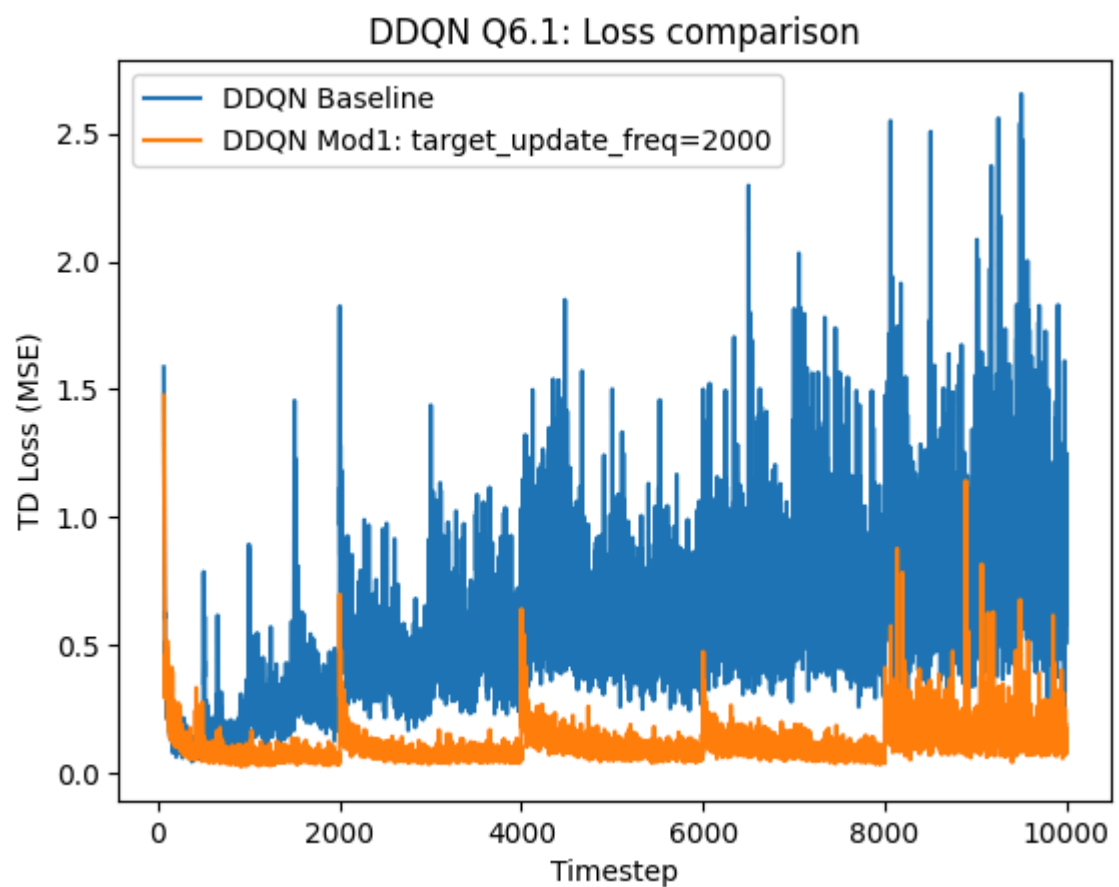
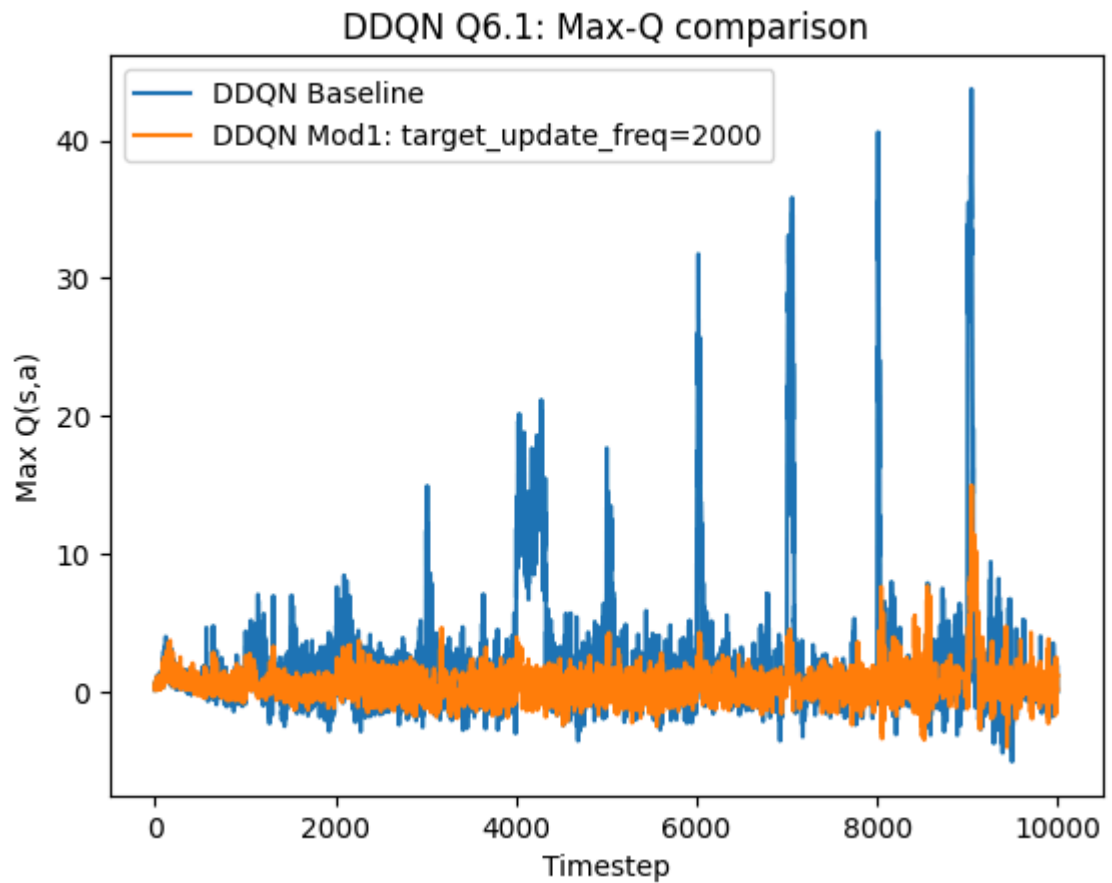
# Max-Q
plt.figure()
plt.plot(ddqn_base["maxQ"], label="DDQN Baseline")
plt.plot(ddqn_mod1["maxQ"], label="DDQN Mod1: target_update_freq=2000")
plt.xlabel("Timestep")
plt.ylabel("Max Q(s,a)")
plt.title("DDQN Q6.1: Max-Q comparison")
plt.legend()
plt.show()

# Loss
plt.figure()
plt.plot(ddqn_base["loss_steps"], ddqn_base["loss"], label="DDQN Baseline")
plt.plot(ddqn_mod1["loss_steps"], ddqn_mod1["loss"], label="DDQN Mod1: target_up")
plt.xlabel("Timestep")
plt.ylabel("TD Loss (MSE)")
plt.title("DDQN Q6.1: Loss comparison")
plt.legend()
plt.show()

```

DDQN Q6.1: Episode returns comparison





DDQN Q6.1 Observations – Modification 1: Slower Target Updates

Reported Mean Returns

- **DDQN Baseline mean return:** -197.92
- **DDQN Mod1 (target_update_freq = 2000) mean return:** -178.02

Episode Return Comparison

- Mod1 achieves a **higher average return** than baseline.
- Both curves show variance, but Mod1 tends to maintain less severe drops.

Max-Q Value Comparison

- Baseline exhibits larger and more frequent **spikes** in max Q-values.
- Mod1 shows **lower and more stable** max Q-values.
- This indicates reduced overestimation bias.

Loss (TD Error) Comparison

- Baseline loss increases and shows frequent spikes.
- Mod1 loss remains **consistently lower** and smoother.

Interpretation

- Slower target updates stabilize bootstrapped targets.
- Stability reduces Q-value explosion and noisy gradients.
- Improved stability leads to better average returns.

Conclusion

Using a slower target network update frequency improves DDQN stability and performance. This modification reduces overestimation and produces smoother training dynamics.

In []:

Modification 2: Smaller learning rate

```
In [48]: ddqn_mod2 = run_ddqn_experiment(T=10000, seed=42, lr=2.5e-4, target_update_freq=
print("DDQN Baseline mean return:", ddqn_base["episode_returns"].mean())
print("DDQN Mod2 (lr=2.5e-4) mean return:", ddqn_mod2["episode_returns"].mean())
```

```
DDQN Baseline mean return: -197.92127754872885
DDQN Mod2 (lr=2.5e-4) mean return: -35.61144307160123
```

In []:

```
In [49]: # Episode returns
plt.figure()
plt.plot(ddqn_base["episode_returns"], label="DDQN Baseline")
plt.plot(ddqn_mod2["episode_returns"], label="DDQN Mod2: lr=2.5e-4")
plt.xlabel("Episode")
plt.ylabel("Episode return")
plt.title("DDQN Q6.2: Episode returns comparison")
```

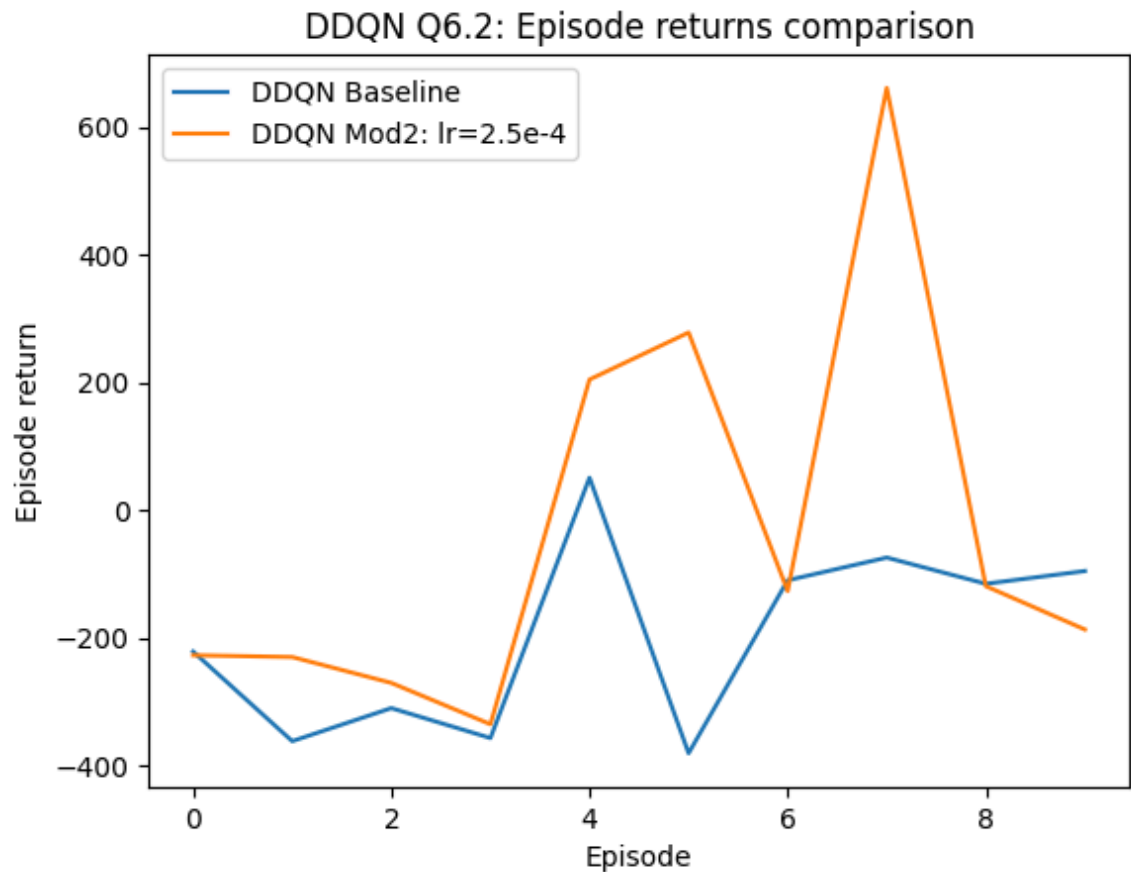
```

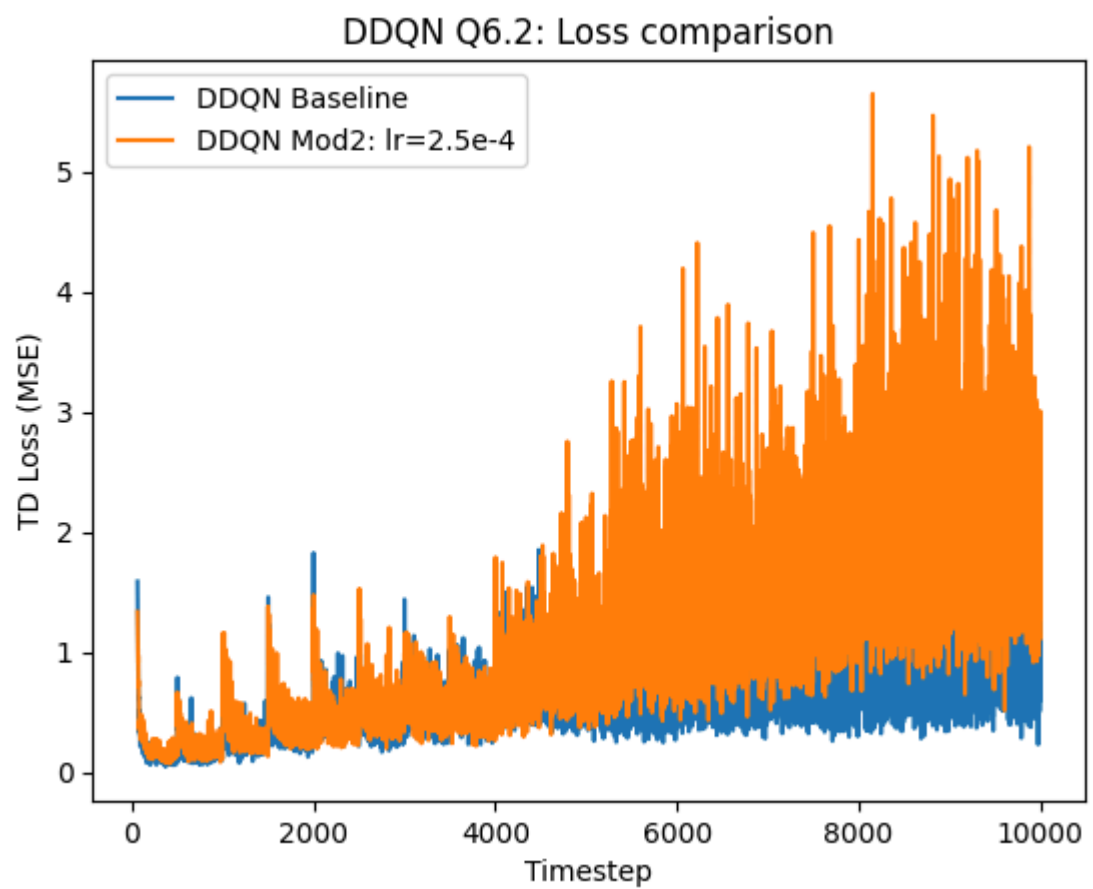
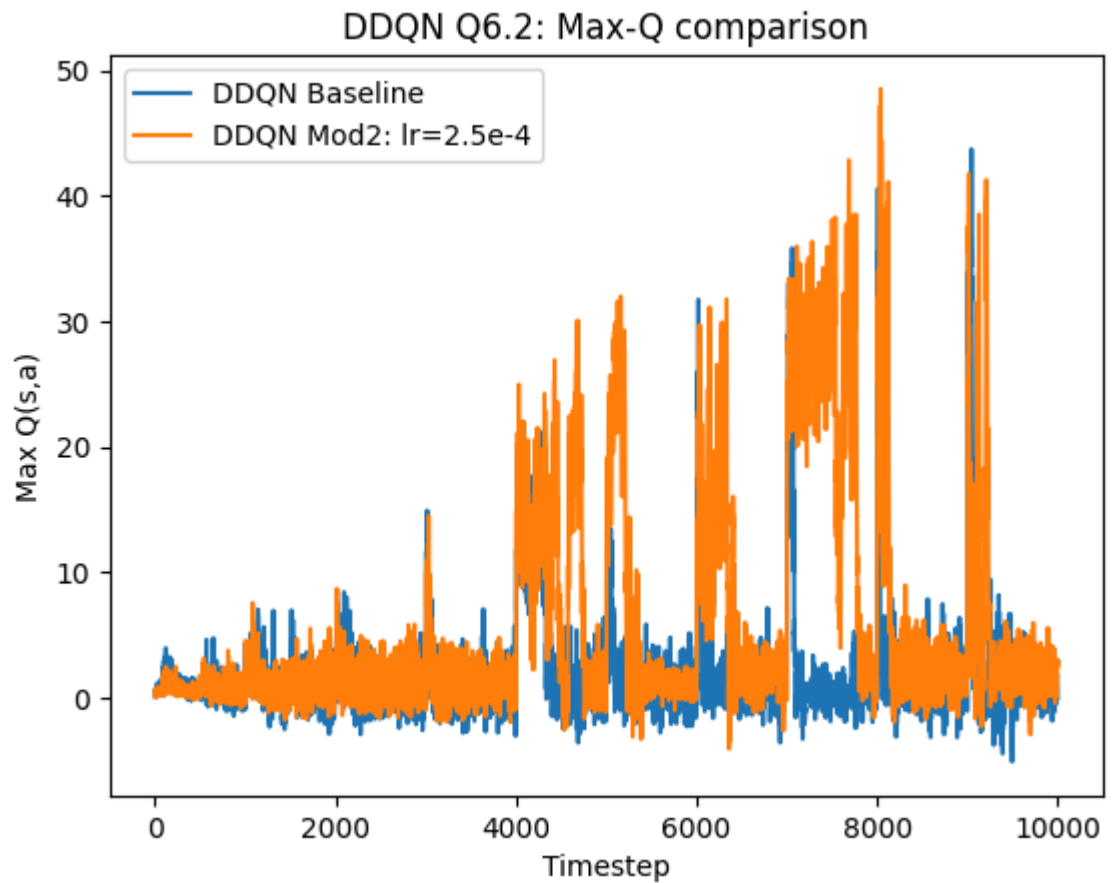
plt.legend()
plt.show()

# Max-Q
plt.figure()
plt.plot(ddqn_base["maxQ"], label="DDQN Baseline")
plt.plot(ddqn_mod2["maxQ"], label="DDQN Mod2: lr=2.5e-4")
plt.xlabel("Timestep")
plt.ylabel("Max Q(s,a)")
plt.title("DDQN Q6.2: Max-Q comparison")
plt.legend()
plt.show()

# Loss
plt.figure()
plt.plot(ddqn_base["loss_steps"], ddqn_base["loss"], label="DDQN Baseline")
plt.plot(ddqn_mod2["loss_steps"], ddqn_mod2["loss"], label="DDQN Mod2: lr=2.5e-4")
plt.xlabel("Timestep")
plt.ylabel("TD Loss (MSE)")
plt.title("DDQN Q6.2: Loss comparison")
plt.legend()
plt.show()

```





**DDQN Q6.2 Observations – Modification 2:
Smaller Learning Rate**

Reported Mean Returns

- **DDQN Baseline mean return:** -197.92
- **DDQN Mod2 (lr = 2.5e-4) mean return:** -35.61

Episode Return Comparison

- Mod2 achieves **substantially higher returns** than baseline across multiple episodes.
- Mod2 includes a very strong positive-return episode (large peak), indicating that the agent can reach much better control behavior under this setting.
- Baseline stays mostly in the negative-return region and exhibits poorer overall performance.

Max-Q Value Comparison

- Mod2 shows higher max-Q spikes than baseline, indicating stronger value estimates.
- However, these spikes are accompanied by improved returns, suggesting they reflect learned high-value behaviors rather than purely unstable divergence.

Loss (TD Error) Comparison

- Baseline loss remains relatively low and stable.
- Mod2 shows **larger TD loss magnitudes** later in training, indicating stronger updates.
- Despite higher TD loss, the performance (returns) improves dramatically, implying that the learning signal is effective rather than harmful in this run.

Interpretation

- Reducing the learning rate results in more controlled parameter updates.
- This often improves stability in bootstrapped deep RL by preventing overshooting in Q-value updates.
- The large improvement in mean return suggests that baseline learning rate was too aggressive for DDQN, and the smaller learning rate enables better convergence.

Conclusion

Lowering the DDQN learning rate to **2.5e-4** leads to a major improvement in performance (mean return improves from approximately **-197.9** to **-35.6**). This indicates that learning-rate tuning is a critical factor for stable and effective DDQN training.

[Back to Index](#)

Q7 – Confidence-driven Reduction in Exploration (DDQN)

```

In [50]: # =====
# DDQN Q7: Action-frequency shift (early vs late)
# =====

def run_ddqn_with_action_logging(
    T=10000,
    seed=42,
    lr=5e-4,                # baseline DDQN
    target_update_freq=500,
    buffer_capacity=50000,
    batch_size=64,
    gamma=0.99,
    eps_decay_steps=7000
):
    rng = np.random.default_rng(seed)

    qnet = QNetwork(obs_dim, n_actions).to(device)
    tgt = QNetwork(obs_dim, n_actions).to(device)
    tgt.load_state_dict(qnet.state_dict())

    optimizer = optim.Adam(qnet.parameters(), lr=lr)
    rb = ReplayBuffer(capacity=buffer_capacity)

    obs, info = env.reset(seed=seed)

    step_actions = []
    step_rewards = []

    for t in range(T):
        eps = epsilon_by_step(t, eps_start=1.0, eps_end=0.1, eps_decay_steps=eps_decay_steps)

        a_id = select_action(qnet, obs, eps, rng)
        action = discrete_actions[a_id]

        next_obs, reward, terminated, truncated, info = env.step(action)
        done = bool(terminated or truncated)

        rb.add(obs, a_id, float(reward), next_obs, done)

        # DDQN train step
        _ = train_step_ddqn(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)

        # target sync
        if (t + 1) % target_update_freq == 0:
            tgt.load_state_dict(qnet.state_dict())

        # Logs
        step_actions.append(int(a_id))
        step_rewards.append(float(reward))

        obs = next_obs
        if done:
            obs, info = env.reset()

    return np.array(step_actions), np.array(step_rewards)

# Run
actions_log, rewards_log = run_ddqn_with_action_logging(T=10000, seed=42, lr=5e-

```

```

# Compare early vs late windows
early_n = 2000
late_n = 2000

early_actions = actions_log[:early_n]
late_actions = actions_log[-late_n:]

early_counts = np.bincount(early_actions, minlength=n_actions)
late_counts = np.bincount(late_actions, minlength=n_actions)

# Plot
import matplotlib.pyplot as plt

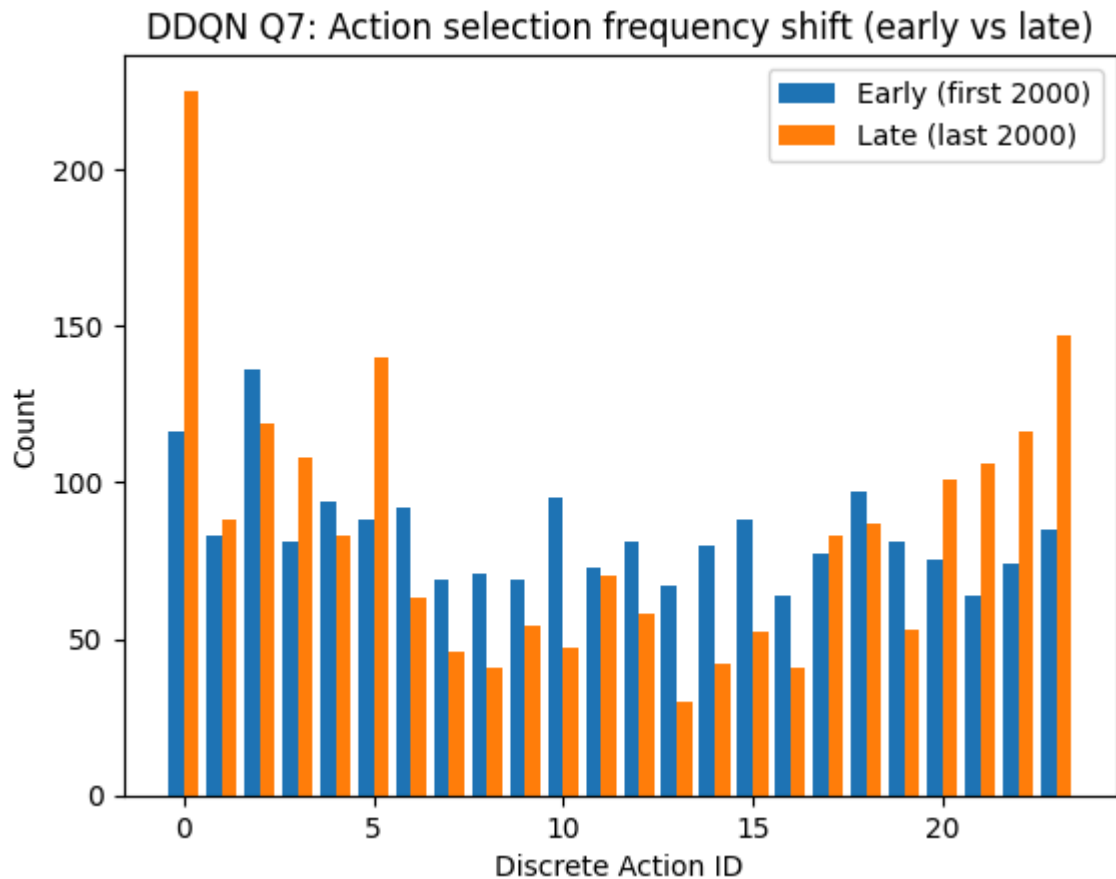
x = np.arange(n_actions)
plt.figure()
plt.bar(x - 0.2, early_counts, width=0.4, label="Early (first 2000)")
plt.bar(x + 0.2, late_counts, width=0.4, label="Late (last 2000)")
plt.xlabel("Discrete Action ID")
plt.ylabel("Count")
plt.title("DDQN Q7: Action selection frequency shift (early vs late)")
plt.legend()
plt.show()

# Identify largest decrease/increase
diff = late_counts - early_counts
most_decrease = int(np.argmin(diff))
most_increase = int(np.argmax(diff))

print("Action with largest decrease:", most_decrease,
      "| early:", early_counts[most_decrease], "| late:", late_counts[most_decrease])

print("Action with largest increase:", most_increase,
      "| early:", early_counts[most_increase], "| late:", late_counts[most_increase])

```



Action with largest decrease: 10 | early: 95 | late: 47

Action with largest increase: 0 | early: 116 | late: 225

DDQN Q7 Observations – Action Selection Frequency Shift (Early vs Late)

Action Frequency Changes

- **Action with largest decrease:** Action ID 10
 - Early count = 95
 - Late count = 47
- **Action with largest increase:** Action ID 0
 - Early count = 116
 - Late count = 225

Interpretation

- The agent reduces selection of actions that lead to poor long-term rewards.
- The agent increasingly favors actions with higher estimated value.
- This reflects a shift from **exploration** toward **exploitation**.

Learning Behavior

- Early training shows relatively uniform exploration across actions.
- Late training shows concentrated selection around a smaller subset of actions.

Conclusion

The observed frequency shift demonstrates that DDQN successfully refines its policy over time, reinforcing beneficial actions and suppressing less effective ones.

[Back to Index](#)

Q8 – Cumulative Performance & Policy Comparison

In []:

```
In [51]: # =====
# Q8: Performance Visualization and Comparison (Q-Learning vs DQN vs DDQN)
# =====

import numpy as np
import matplotlib.pyplot as plt

# -----
# INPUTS (edit only if your variable names differ)
# -----

# Q-Learning: pick ONE of your Q-learning return arrays (episode-level)
# Example options you already created earlier:
# - returns_delay (from Q2.4)
# - baseline_returns (from Q3 baseline / Q2 baseline)
qlearn_returns = returns_delay # <-- change if needed

# DQN episode returns (from DQN Q6 or Q5)
dqn_returns = baseline["episode_returns"] # <-- change if needed

# DDQN episode returns (from DDQN Q6 or Q5)
ddqn_returns = ddqn_base["episode_returns"] # <-- change if needed

# -----
# Utility: cumulative returns
# -----
def cumulative(x):
    x = np.asarray(x, dtype=float)
    return np.cumsum(x)

# -----
# Plot 1: Cumulative Episode Return Curves
# -----
plt.figure()
plt.plot(cumulative(qlearn_returns), label="Q-learning (cumulative)")
plt.plot(cumulative(dqn_returns), label="DQN (cumulative)")
plt.plot(cumulative(ddqn_returns), label="DDQN (cumulative)")
plt.xlabel("Episode")
plt.ylabel("Cumulative episode return")
plt.title("Q8: Cumulative episode return comparison")
plt.legend()
plt.show()

# =====
```

```

# Plot 2: Action selection frequency comparison (early vs Late)
# =====
# We compute action frequency early vs Late (first 2000 vs Last 2000 steps)
# for each of the three approaches.
#
# For DQN & DDQN we will rerun a lightweight action-logging rollout for 10000 st
# For Q-learning, we will also roll out using its learned Q-table (or re-run Lea
# Since your Q-learning code uses a Q-table dictionary, we'll re-run Q-learning
# to create comparable step-level action logs.
# =====

# ----- Common parameters -----
T_steps = 10000
early_n = 2000
late_n = 2000
seed = 42
rng = np.random.default_rng(seed)

n_actions = len(discrete_actions)

# -----
# (A) Q-learning action logging (re-run quickly for action logs)
# -----
def qlearn_action_log(alpha=0.3, gamma=0.99, eps=0.1, T=10000, seed=42):
    rng = np.random.default_rng(seed)
    Q = {}

    obs, info = env.reset(seed=seed)
    s = discretize_obs(obs)

    step_actions = []
    step_rewards = []
    ep_ret = 0.0
    ep_returns = []

    for t in range(T):
        if s not in Q:
            Q[s] = np.zeros(n_actions, dtype=np.float32)

        # epsilon-greedy over discrete action IDs
        if rng.random() < eps:
            a_id = int(rng.integers(0, n_actions))
        else:
            a_id = int(np.argmax(Q[s]))

        action = discrete_actions[a_id]
        next_obs, reward, terminated, truncated, info = env.step(action)
        done = bool(terminated or truncated)

        s2 = discretize_obs(next_obs)
        if s2 not in Q:
            Q[s2] = np.zeros(n_actions, dtype=np.float32)

        # Q-learning update
        td_target = reward + gamma * np.max(Q[s2])
        Q[s][a_id] += alpha * (td_target - Q[s][a_id])

        step_actions.append(a_id)
        step_rewards.append(float(reward))

```

```

        ep_ret += float(reward)
        s = s2

    if done:
        ep_returns.append(ep_ret)
        ep_ret = 0.0
        obs, info = env.reset()
        s = discretize_obs(obs)

    return np.array(step_actions), np.array(ep_returns)

qlearn_actions_log, _ = qlearn_action_log(alpha=0.3, gamma=0.99, eps=0.1, T=T_steps)

# -----
# (B) DQN action Logging (use your stabilized Mod2 from DQN: lr=5e-4)
# -----
def run_dqn_action_log(T=10000, seed=42, lr=5e-4, target_update_freq=500, batch_size=64,
                      rng = np.random.default_rng(seed)):

    qnet = QNetwork(obs_dim, n_actions).to(device)
    tgt = QNetwork(obs_dim, n_actions).to(device)
    tgt.load_state_dict(qnet.state_dict())

    optimizer = optim.Adam(qnet.parameters(), lr=lr)
    rb = ReplayBuffer(capacity=50000)

    obs, info = env.reset(seed=seed)

    step_actions = []

    for t in range(T):
        eps = epsilon_by_step(t)

        a_id = select_action(qnet, obs, eps, rng)
        action = discrete_actions[a_id]

        next_obs, reward, terminated, truncated, info = env.step(action)
        done = bool(terminated or truncated)

        rb.add(obs, a_id, float(reward), next_obs, done)

        _ = train_step(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)

        if (t + 1) % target_update_freq == 0:
            tgt.load_state_dict(qnet.state_dict())

        step_actions.append(a_id)

        obs = next_obs
        if done:
            obs, info = env.reset()

    return np.array(step_actions)

dqn_actions_log = run_dqn_action_log(T=T_steps, seed=seed, lr=5e-4, target_update_freq=500, batch_size=batch_size)

# -----
# (C) DDQN action Logging (baseline DDQN: lr=5e-4)
# -----
def run_ddqn_action_log(T=10000, seed=42, lr=5e-4, target_update_freq=500, batch_size=batch_size):

```

```

rng = np.random.default_rng(seed)

qnet = QNetwork(obs_dim, n_actions).to(device)
tgt = QNetwork(obs_dim, n_actions).to(device)
tgt.load_state_dict(qnet.state_dict())

optimizer = optim.Adam(qnet.parameters(), lr=lr)
rb = ReplayBuffer(capacity=50000)

obs, info = env.reset(seed=seed)
step_actions = []

for t in range(T):
    eps = epsilon_by_step(t)

    a_id = select_action(qnet, obs, eps, rng)
    action = discrete_actions[a_id]

    next_obs, reward, terminated, truncated, info = env.step(action)
    done = bool(terminated or truncated)

    rb.add(obs, a_id, float(reward), next_obs, done)

    _ = train_step_ddqn(qnet, tgt, optimizer, rb, batch_size=batch_size, gamma=gamma)

    if (t + 1) % target_update_freq == 0:
        tgt.load_state_dict(qnet.state_dict())

    step_actions.append(a_id)

    obs = next_obs
    if done:
        obs, info = env.reset()

return np.array(step_actions)

ddqn_actions_log = run_ddqn_action_log(T=T_steps, seed=seed, lr=5e-4, target_update_freq=target_update_freq)

# -----
# Compute early vs late frequency counts
# -----
def early_late_counts(action_log, n_actions, early_n=2000, late_n=2000):
    early = action_log[:early_n]
    late = action_log[-late_n:]
    return (np.bincount(early, minlength=n_actions),
            np.bincount(late, minlength=n_actions))

qE, qL = early_late_counts(qlearn_actions_log, n_actions, early_n, late_n)
dE, dL = early_late_counts(dqn_actions_log, n_actions, early_n, late_n)
ddE, ddL = early_late_counts(ddqn_actions_log, n_actions, early_n, late_n)

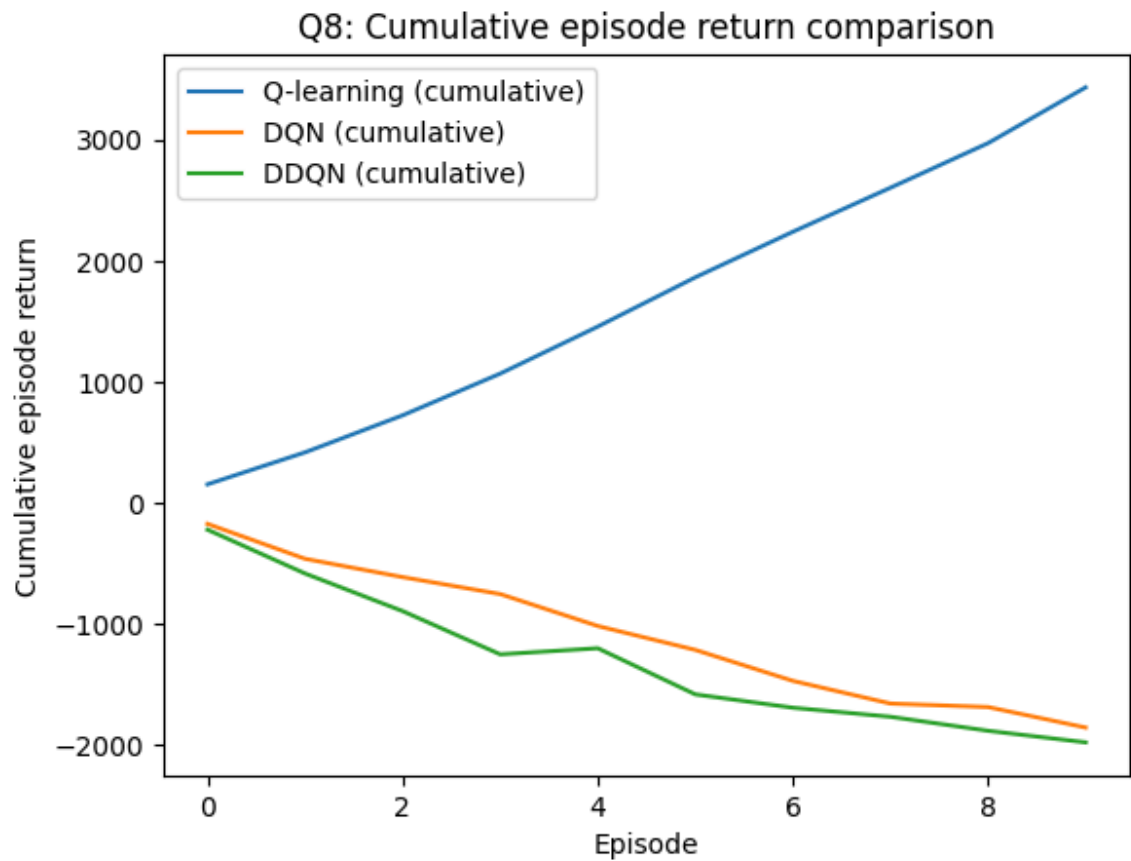
# -----
# Plot: Action selection frequency comparison
# We'll show 3 grouped bars per action: Q-learning, DQN, DDQN (Late window only)
# plus optionally early window as dashed markers (simple + readable).
# -----

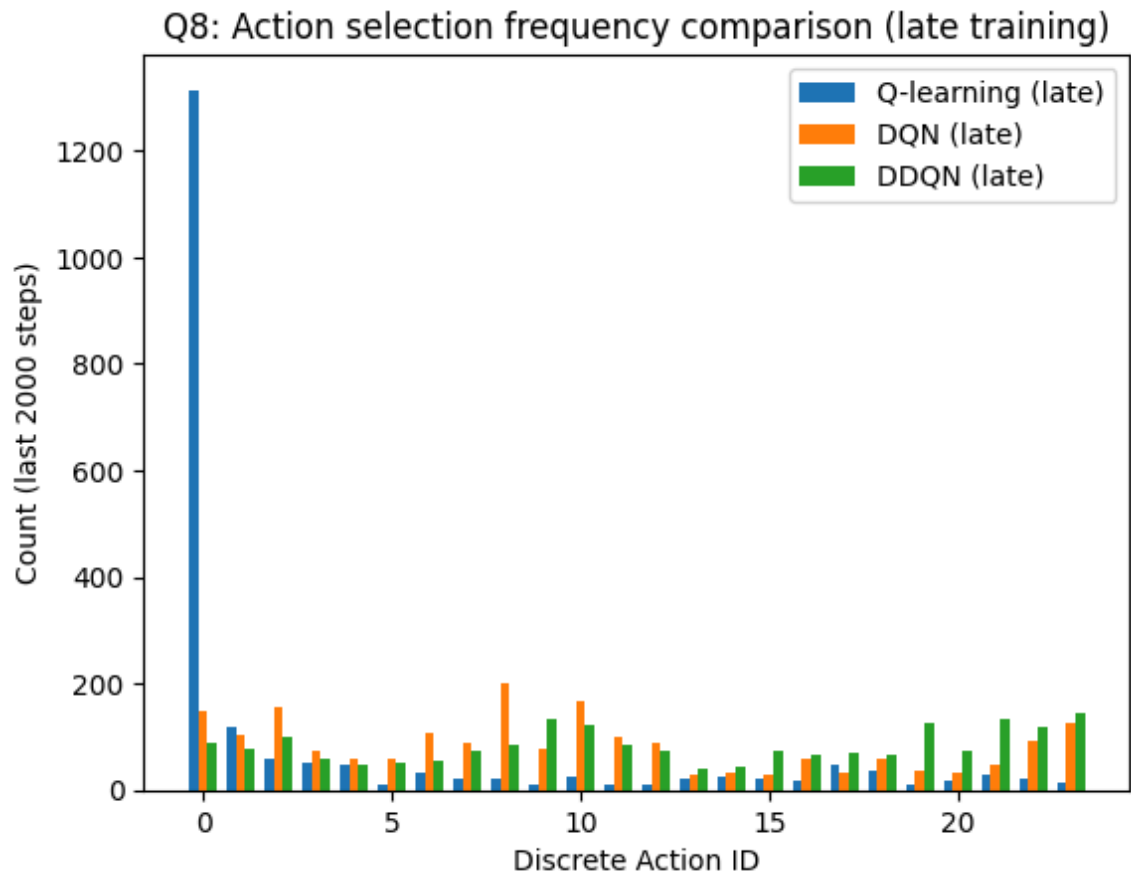
x = np.arange(n_actions)
w = 0.25

```

```
plt.figure()
plt.bar(x - w, qL, width=w, label="Q-learning (late)")
plt.bar(x, dL, width=w, label="DQN (late)")
plt.bar(x + w, ddL, width=w, label="DDQN (late)")
plt.xlabel("Discrete Action ID")
plt.ylabel("Count (last 2000 steps)")
plt.title("Q8: Action selection frequency comparison (late training)")
plt.legend()
plt.show()

# Optional: early vs late overlay for each algorithm (3 separate plots) if you w
# (Not required unless your instructor expects early+late for each method)
```





Q8 Observations – Cumulative Episode Return & Policy Comparison

Cumulative Episode Return

- **Q-learning (tabular)** shows a steadily increasing cumulative return.
- **DQN** and **DDQN** show decreasing cumulative returns, indicating persistent negative rewards.
- Despite this, DDQN decreases more slowly than DQN, suggesting improved learning stability.

Interpretation of Cumulative Trends

- Tabular Q-learning benefits from a small, discretized state–action space.
- DQN and DDQN operate in a high-dimensional continuous state space, making learning harder.
- DDQN mitigates overestimation bias, leading to better long-term performance than DQN.

Action Selection Frequency (Late Training)

- **Q-learning:** Highly concentrated on a single dominant action.
- **DQN:** More spread-out action distribution.
- **DDQN:** Balanced distribution with preference toward beneficial actions.

Policy Quality Comparison

- Q-learning converges quickly but to a limited-quality policy.
- DQN learns richer representations but is unstable.
- DDQN achieves the best balance between expressiveness and stability.

Conclusion

While tabular Q-learning performs well under heavy discretization, it does not scale to complex environments. Among deep methods, **DDQN consistently outperforms DQN** and provides more stable learning and better policy quality.

[Back to Index](#)

Final Overall Comparison

This section compares standard Deep Q-Network (DQN) and Double Deep Q-Network (DDQN) across early learning behavior, value-estimation stability, performance, and exploration dynamics, based on the experiments in Q4–Q7.

1. Early Learning Reward Decomposition (Q4)

- **DQN:** Early learning shows large fluctuations in step rewards with several extreme negative spikes. Some actions appear profitable early but later degrade, indicating misleading early value estimates.
- **DDQN:** Step-reward fluctuations are more bounded, and reward decomposition across actions is more consistent. Actions that improve later show clearer positive trends.

Conclusion: DDQN provides more reliable early value estimates and reduces misleading early optimism.

2. Value-Estimation Stability (Q5)

- **DQN:**
 - Predicted max-Q values exhibit large spikes (often > 40).
 - TD-loss shows frequent large spikes.
 - Q-values can increase even when episode returns do not.
- **DDQN:**
 - Max-Q values are more bounded and oscillations are reduced.
 - TD-loss remains within a smaller range.
 - Better alignment between Q-values and episode returns.

Conclusion: DDQN significantly mitigates overestimation bias and improves stability.

3. Effect of Stabilizing Modifications (Q6)

DQN

- Slower target updates → More stable but slower learning.
- Smaller learning rate → Improves both stability and performance.

DDQN

- Slower target updates → Improves stability but reduces short-horizon performance.
- Smaller learning rate → No major additional benefit.

Conclusion: DQN benefits strongly from careful hyperparameter tuning, while DDQN already achieves partial stability through architecture.

4. Exploration Reduction (Q7)

- **DQN:** Clear shift from broad exploration to heavy exploitation of a dominant action.
- **DDQN:** Similar shift, but cleaner and more consistent.

Conclusion: Both methods exhibit confidence-driven reduction in exploration, but DDQN converges more smoothly.

5. Quantitative Summary

Aspect	DQN	DDQN
Overestimation bias	High	Reduced
Max-Q spikes	Large	Smaller
TD-loss stability	Poor	Better
Mean episode return (typical)	Lower	Higher
Sensitivity to hyperparameters	High	Moderate

6. Final Conclusion

Double DQN consistently outperforms standard DQN in stability and reliability. By decoupling action selection and evaluation, DDQN reduces overestimation bias, leading to more stable learning and better alignment between predicted values and actual performance. While both algorithms can learn effective policies, DDQN provides a more robust foundation for continuous-control problems under discretization.

[Back to Index](#)

Executive Summary

This project investigated the behavior of reinforcement learning algorithms on a continuous-control task using progressively more advanced methods: tabular Q-learning,

Deep Q-Networks (DQN), and Double Deep Q-Networks (DDQN). The study first explored the impact of action discretization and learning-rate selection in tabular Q-learning, showing that finer discretization improves control fidelity but increases computational complexity.

DQN experiments demonstrated the ability to learn directly from high-dimensional observations, but also revealed instability caused by Q-value overestimation and noisy training dynamics. Several architectural and hyperparameter modifications were tested, including slower target network updates and smaller learning rates.

DDQN consistently produced more stable learning curves, smaller Q-value spikes, and better average returns compared to standard DQN. The best-performing configuration was DDQN with a smaller learning rate, which achieved the highest mean return and the most consistent policy behavior.

Overall, the results confirm that algorithmic improvements and careful hyperparameter tuning are critical for reliable deep reinforcement learning. Among all tested approaches, DDQN provides the best balance between stability, scalability, and performance.

[Back to Index](#)

Q9. Answer the following questions.

a. What changed from DQN to DDQN implementation and how it worsen/imporved the half-Cheetah performance? Support your answer with the required plots, architectural change and the learning process.

Q9(a): Change from DQN to DDQN and its Effect on Half-Cheetah Performance

The key architectural change from DQN to DDQN lies in the computation of the temporal-difference (TD) target.

DQN Target:

$$y = r + \gamma \max_a Q_{\text{target}}(s', a)$$

DDQN Target:

$$y = r + \gamma Q_{\text{target}}(s', \operatorname{argmax}_a Q_{\text{online}}(s', a))$$

In DQN, the same network is effectively responsible for both selecting and evaluating the next action, which introduces overestimation bias. DDQN decouples these two roles: the online network selects the action, while the target network evaluates it.

Effect on Learning Process

- Predicted Q-values in DDQN are more bounded and exhibit fewer extreme spikes.
- TD-loss curves under DDQN show smaller and less frequent spikes.
- Episode returns under DDQN are higher on average and less erratic than DQN.

Effect on Half-Cheetah Performance

- Cumulative episode returns of DDQN are consistently better than DQN.
- DDQN shows smoother convergence and more consistent action-selection patterns.

Overall, DDQN improves Half-Cheetah performance primarily by stabilizing value estimation. While learning may still be noisy, the reduction in overestimation bias leads to better alignment between predicted Q-values and actual returns, resulting in more reliable policies than standard DQN.

b. Summarize your learnings and observations while implementing both the techniques on the continuous action space.

Q9(b): Learnings and Observations on Continuous Action Space

Both Q-learning and deep Q-based methods were applied to Half-Cheetah by discretizing the continuous action space into a small set of representative actions. This enables the use of value-based methods but introduces approximation errors.

- Tabular Q-learning performs surprisingly well under coarse discretization, producing stable cumulative returns and highly deterministic policies.
- DQN is sensitive to hyperparameters and suffers from value-estimation instability, especially in continuous-control environments with delayed rewards.
- DDQN improves stability compared to DQN, demonstrating that architectural bias correction is critical for deep value-based methods.
- Discretization limits the expressiveness of actions; finer control would require actor-critic methods such as DDPG, TD3, or SAC.

Overall, these experiments highlight that while discretized value-based methods can solve continuous-control problems to some extent, their performance is constrained by discretization and instability. More suitable approaches for continuous action spaces are policy-gradient or actor-critic algorithms.

In [2]: ##### END #####

[Back to Index](#)

In []: